

I/O

I dispositivi di I/O sono vari:

- Memorizzazione (SSD, dischi)
- Trasmissione dei dati (schede di rete)
- Interfaccia utente (tastiera, mouse)

I/O in parte è **competenza del SO** (controlla i dispositivi e fornisce un modo uniforme per accedervi da parte dei programmi) e in **parte hw**.

DEVICE DRIVER = **componente sw**, solitamente fornito dal produttore dell'hw, che **contiene i dettagli del dispositivo e ne conosce il funzionamento**. Viene **ESEGUITO dal SO** (il firmware invece è il sw interno al dispositivo). Fornisce al sottosistema I/O del SO un'interfaccia uniforme in modo che il kernel può richiedere una lettura/scrittura del dispositivo senza conoscere in dettaglio la periferica.

I dispositivi comunicano con il computer attraverso **porte**, **bus**, **controller**.

Il driver comunica con il dispositivo tramite alcuni registri hw: data-in-register, data-out-register, status register, control register. Per **accedere** a questi registri si usano **2 tecniche**:

- **DIRECT I/O** □ il processore usa **istruzioni di I/O speciali** e uno **spazio di indirizzamento separato da quello della memoria**. Il dispositivo viene **identificato tramite un indirizzo di porta e raggiunto con queste istruzioni speciali**.
- **MEMORY-MAPPED I/O** □ non c'è uno spazio di indirizzamento separato dalla memoria, ma **i data register / control register del dispositivo hanno assegnati degli indirizzi nello stesso spazio di indirizzamento usato dal processore per la memoria e vi si accede usando normali istruzioni di lettura/scrittura**. Soluzione molto conveniente per spazi di indirizzamento molto grandi o per dispositivi con grandi quantità di dati (schede grafiche).

La **comunicazione tra CPU e dispositivi di I/O** può essere gestita tramite:

- POLLING
- INTERRUPT
- DMA

POLLING

Per ogni trasferimento **la CPU legge il busy bit dello status register**, fino a che non è = 0 (dispositivo è libero). Poi **imposta l'operazione di lettura / scrittura**, **inserisce il dato nel data-out register e attiva il command-ready bit**. Imposta il **busy bit = 1**, fa il **trasferimento dati** e infine setta: busy bit = 0, error bit = 0, command-ready-bit = 0. FA **BUSY WAIT**. Va bene per **dispositivi molto veloci**, ma non per quelli lenti perché la CPU rimane occupata ad attendere e non può fare altro.

INTERRUPT

Dopo aver avviato l'operazione, la CPU può fare altro, eseguire altre istruzioni. Gli **interrupt** possono avere **priorità diverse**: **mascherabili** (temporaneamente ignorabili), **non mascherabili** (segnalare eventi critici). Quando + dispositivi condividono stesso numero di interrupt, l'handler li controlla in sequenza con **interrupt chaining**.

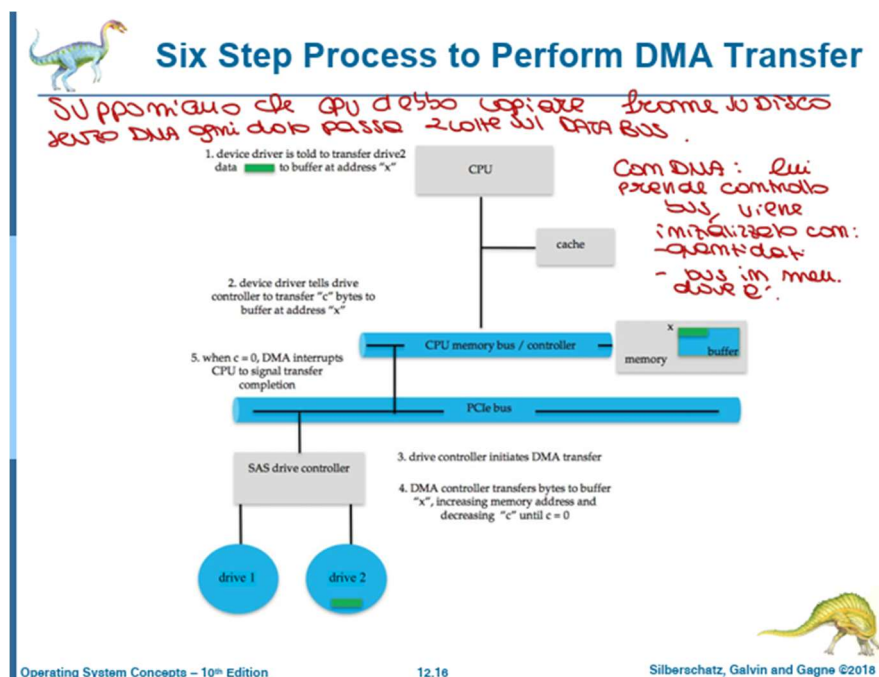
Il **device driver** avvia l'operazione di I/O scrivendo i comandi nei registri del controller. Il controller prende in carico la richiesta e imposta il dispositivo come occupato tramite il **busy bit**. A differenza del polling, la CPU non controlla continuamente lo stato del dispositivo: prosegue l'esecuzione di altre istruzioni, verificando gli interrupt tra un'istruzione e la successiva. Quando il trasferimento termina, oppure si verifica un errore, il controller genera **un segnale di interrupt**. La CPU completa l'istruzione corrente, salva il contesto del programma interrotto e trasferisce il controllo all'**interrupt handler** associato a quel dispositivo mediante l'**interrupt vector**. L'handler acquisisce i dati, verifica lo stato dell'operazione, gestisce eventuali errori e conferma la ricezione dell'interrupt. Infine ripristina il contesto salvato ed esegue il **return from interrupt**, permettendo alla CPU di riprendere il programma esattamente dal punto in cui era stato interrotto. Questo metodo **elimina la busy wait**: la CPU attende comunque il completamento logico dell'I/O, ma nel frattempo può svolgere altro lavoro. Il segnale di interrupt può essere mantenuto attivo dal **controller** finché la CPU non lo riconosce, evitando che la notifica venga persa.

DMA (Direct Memory Access)

Viene utilizzato per evitare l'**I/O programmato**, nel quale la CPU trasferisce i dati un byte alla volta. È quindi conveniente quando bisogna spostare grandi quantità di dati. Per funzionare richiede un **controller DMA**, che trasferisce i dati direttamente tra il dispositivo di I/O e la memoria, senza farli passare attraverso la CPU. Il SO prepara in memoria un **DMA command block**, che contiene:

- indirizzo sorgente e indirizzo destinazione;
- modalità di lettura o scrittura;
- numero di byte da trasferire.

Successivamente comunica al controller DMA l'indirizzo in memoria di questo blocco di comando. Il controller DMA legge le informazioni e prende temporaneamente il controllo del bus al posto della CPU: questa capacità è chiamata **bus mastering**. Durante il trasferimento può sottrarre alla CPU alcuni cicli di accesso al bus, fenomeno chiamato **cycle stealing**, ma resta più efficiente rispetto all'I/O programmato. Quando termina, il DMA genera un interrupt per avvisare la CPU. Il **DVMA** è una versione più efficiente capace di utilizzare indirizzi virtuali.



Supponiamo che la CPU debba copiare c byte dal disco 2 a un buffer della memoria RAM che comincia dall'indirizzo x . Senza DMA, la CPU dovrebbe trasferire personalmente ogni dato attraverso il bus; con il DMA, prepara l'operazione e lascia che sia il controller DMA a eseguirla.

Il trasferimento avviene così:

1. Il **device driver** riceve la richiesta di trasferire i dati dal disco 2 al buffer che parte da x .
2. Il driver comunica al **controller SAS del disco** quali dati leggere, l'indirizzo x e il numero di byte c .
3. Il controller del disco avvia il **controller DMA**.
4. Il DMA trasferisce direttamente i dati dal disco alla RAM. Dopo ogni byte aumenta x , per indicare la cella di memoria successiva, e diminuisce c , per indicare quanti byte restano.
5. Quando $c = 0$, il trasferimento è terminato e il **DMA invia un interrupt alla CPU**.

Quindi la CPU non copia i dati: configura l'operazione iniziale e viene avvisata soltanto alla fine.

Sì, **DMA e interrupt vengono normalmente usati insieme**, ma hanno funzioni diverse:

- Il **DMA esegue il trasferimento** dei dati direttamente tra dispositivo e memoria, senza coinvolgere la CPU per ogni byte.
- L'**interrupt avvisa la CPU** quando il trasferimento è terminato oppure se si è verificato un errore.

CYCLE STEALING = durante un trasferimento DMA, il DMA controller deve utilizzare il bus di sistema per trasferire i dati tra il dispositivo di I/O e la RAM. Ma lo stesso bus viene anche utilizzato dalla CPU per accedere alla memoria. Quindi il DMA controller prende temporaneamente il controllo del bus per uno /alcuni cicli e trasferisce una parte di dati. In quei cicli la CPU non può usare il bus e deve attendere, però può continuare ad eseguire istruzioni che non richiedono il bus (no op. Di memoria).

Con il DMA evitiamo il doppio trasferimento dispositivo-CPU e CPU-RAM. La CPU può eseguire altri processi durante il trasferimento. Se ad es dobbiamo trasferire 10MB, con il DMA i dati passano direttamente dal controller del disco alla RAM e quindi ogni byte attraversa il bus una sola volta: 10MB. Con I/O programmato ogni dato compie 2 trasferimenti: 1) dal controller del disco ad un registro della CPU, 2) dal registro della CPU alla RAM. Ogni byte quindi transita 2 volte: $2 \times 10 \text{ MB} = 20 \text{ MB}$. Se con il DMA usassimo anche un BUFFER KERNEL : dispositivo → DMA → buffer Kernel → copia CPU → memoria USER, allora il DMA controller esegue lo stesso numero di trasferimenti e trasferisce la stessa quantità di dati. Cambia solo l'indirizzo di destinazione (anziché la memoria user, avremmo la memoria kernel). Poi la CPU deve fare una copia aggiuntiva tra memoria kernel e memoria user ma questa copia non è fatta dal DMA. In questo caso il lavoro del DMA rimane invariato, ma il traffico complessivo sulla memoria e l'utilizzo della CPU sono maggiori.

I programmi usano le **system call** come interfaccia per comunicare con i dispositivi (descrivono operazioni generiche). Il kernel poi inoltra la richiesta al device driver che traduce l'operazione generica in comandi specifici del controller, **il driver nasconde le differenze hw tra i dispositivi**. **Il controller è hw che esegue concretamente i comandi**. Tutti i dispositivi NON hanno lo stesso driver.

Un dispositivo può essere contemporaneamente: a blocchi; ad accesso casuale; condiviso; sincrono o asincrono nell'uso; molto veloce; leggibile e scrivibile.

Queste proprietà non si escludono e non implicano automaticamente le altre.

Character stream o block

Questa distinzione riguarda come sono **organizzati i dati**.

- **Character device:** produce o riceve un **flusso di byte**, senza blocchi indirizzabili separatamente. Esempi: tastiera, mouse, porta seriale.
- **Block device:** conserva dati in **blocchi identificabili e indirizzabili**. Esempi: disco e SSD.

Trabocchetto: “character” non significa necessariamente testo; significa flusso di byte.

Un processo user vuole effettuare un input da un dispositivo di IO a caratteri (character device) mediante una strategia basata su polling del dispositivo. Per evitare un'attesa troppo lunga (nel loop di polling) da parte del driver del dispositivo, si chiede all'autore del programma eseguito dal processo user, di fare direttamente polling mediante un loop di lettura del registro di stato del dispositivo.

RICORDA: un processo user non può leggere direttamente i registri del dispositivo perché l'accesso all'hw richiede i privilegi kernel. Il processo deve usare una SYSTEM CALL, sarà poi il driver in modalità kernel a effettuare il POLLING.

Supponendo che il dispositivo sia associato alla tastiera e lo si gestisca in interrupt, si genera un interrupt per ogni carattere ricevuto oppure uno per ogni “Invio”? Si genera un INTERRUPT per ogni CARATTERE/EVENTO ricevuto. La tastiera è un dispositivo a caratteri. L'accumulo dei caratteri fino alla pressione di Invio viene gestito successivamente dal software tramite un buffer.

È possibile che una sola lettura, mediante read(), tenti di acquisire dal dispositivo un vettore di dimensione maggiore di un blocco? Sì. La dimensione richiesta da read() non è limitata alla dimensione del blocco. Se il vettore è più grande di un blocco, il sistema effettua la lettura utilizzando più blocchi

Dato il driver che realizza la lettura, ci si aspetta che la realizzi mediante DMA, qualora possibile, oppure no? Perché → Sì se si può tramite DMA perché. è più efficiente, evitiamo di far transitare i dati nei registri della CPU.

Sequential o random access

Questa distinzione riguarda **come si raggiunge un dato**.

- **Sequential access:** bisogna seguire l'ordine del flusso (bisogna accedere alle precedenti).
- **Random access:** si può raggiungere direttamente una posizione.

Il **disco** è un **block device** con **accesso casuale**, perché può leggere direttamente un determinato blocco. La tastiera è sequenziale perché i caratteri arrivano in ordine.

Trabocchetto: “block” e “random access” sono concetti distinti, anche se spesso i dischi possiedono entrambe le proprietà.

Sharable o dedicated

Questa distinzione riguarda **chi può usare il dispositivo**.

- **Sharable:** può servire richieste provenienti da più processi.
- **Dedicated:** viene assegnato esclusivamente a un processo.

“Condiviso” non significa che il dispositivo esegua più operazioni nello stesso istante: il kernel può mettere le richieste in coda.

Block device e character device

I block device supportano tipicamente:

- `read()` per leggere;
- `write()` per scrivere;
- `seek()` per selezionare una posizione.

Possono essere usati:

- tramite **raw/direct I/O**, accedendo direttamente ai blocchi del dispositivo;
- **tramite il file system**, usando file e directory;
- mediante **memory-mapped file**, associando il file alla memoria virtuale.

Nel memory-mapped file, quando il processo accede a una pagina non presente in RAM, avviene un page fault e il sistema la carica dal disco tramite **demand paging**.

Attenzione al trabocchetto:

- **Memory-mapped file**: un file è associato alla memoria virtuale.
- **Memory-mapped I/O**: i registri di un dispositivo sono associati agli indirizzi della CPU.

Sono due concetti diversi.

I character device gestiscono invece un flusso tramite operazioni concettualmente simili a **get()** e **put()**. Funzioni come l'editing di una riga sono aggiunte da livelli software superiori, non dall'hardware.

Blocking e nonblocking: quando ritorna la chiamata?

Questa distinzione riguarda il comportamento della **system call nel momento in cui viene eseguita**.

Blocking I/O

Se i dati non sono disponibili, il processo viene sospeso fino a quando l'operazione può terminare. Esempio: il processo chiama `read()` sulla tastiera, ma nessun tasto è stato premuto. Il processo dorme fino all'arrivo del dato. Trabocchetto fondamentale:

È bloccato il processo, non la CPU. La CPU può eseguire un altro processo.

Nonblocking I/O

La chiamata non aspetta: ritorna subito con ciò che è disponibile. Esempio: vengono richiesti 100 byte, ma ne sono disponibili 20. La chiamata può restituire immediatamente quei 20 byte. Se non è disponibile nulla, **segnala che in quel momento non può trasferire dati.** `select()` serve a verificare quali descrittori sono pronti prima di chiamare `read()` o `write()`. Il **multithreading** può rendere l'applicazione complessivamente reattiva: un thread può attendere l'I/O mentre gli altri continuano. Ma attenzione: la chiamata del singolo thread può comunque essere bloccante.

«Se un'applicazione multithread usa una `read()` bloccante, l'intera applicazione viene necessariamente sospesa.» □ FALSA

«Il multithreading può mantenere reattiva un'applicazione anche quando un thread esegue I/O bloccante.» □ VERO

Attenzione: il sistema non deve necessariamente completare in background la richiesta originale dei 100 byte. Il processo potrebbe dover riprovare.

Sincrono e asincrono: chi attende il completamento?

Quindi sincrono non significa necessariamente bloccante. Può essere sia blocking che non-blocking.

L'I/O asincrono NON è bloccante!

Questa distinzione riguarda il rapporto tra il processo e il **completamento dell'intera operazione**.

I/O sincrono bloccante

Il processo esegue una system call di I/O, per esempio `read()`. Se i dati non sono ancora disponibili, il kernel sospende il processo e lo mette nello stato `waiting/blocked`. La CPU non rimane ferma, ma può eseguire altri processi. Quando il dispositivo completa il trasferimento, il controller genera un interrupt; l'interrupt handler gestisce il completamento e il kernel sposta il processo da `waiting` a `ready`. Quando lo scheduler gli assegna nuovamente la CPU, la system call ritorna con i dati e il processo continua. È sincrono perché la chiamata ritorna dopo il trasferimento ed è bloccante perché il processo viene sospeso durante l'attesa.

I/O sincrono non bloccante

Il processo esegue una system call, ma se i dati non sono disponibili la chiamata ritorna immediatamente senza sospenderlo, per esempio restituendo `EAGAIN`. Se è disponibile solo una parte dei dati, può restituire soltanto quella parte. Il kernel non mantiene necessariamente attiva l'intera richiesta in background: il processo deve riprovare successivamente oppure usare `select()` per controllare quando il descrittore è pronto. È sincrono perché ogni chiamata restituisce un risultato già determinato in quel momento — dati trasferiti, trasferimento parziale o indisponibilità — ed è non bloccante perché il processo non viene sospeso.

I/O asincrono

Il processo invia una richiesta di I/O e la chiamata ritorna immediatamente, anche se il trasferimento non è ancora terminato. In questo caso, però, diversamente dall'I/O non bloccante, il kernel mantiene attiva la richiesta e continua a eseguirla in background. Il processo può quindi svolgere altro lavoro senza ripetere la system call. Quando il dispositivo termina, il controller genera un interrupt verso il kernel; l'interrupt handler gestisce il completamento e il kernel notifica il processo tramite un segnale, un evento o un callback. Solo dopo questa notifica il risultato può essere considerato disponibile. È

asincrono perché la richiesta iniziale e il completamento avvengono in due momenti separati. Difficile da usare.

Tipologia	Cosa succede al processo	Quando ritorna la chiamata	Se i dati non sono pronti	Chi controlla il completamento	Come riceve il risultato
Sincrono bloccante	Viene sospeso e passa in stato waiting	Quando l'I/O può essere completato (completato il trasferimento dei dati)	Il processo aspetta	Il kernel gestisce l'I/O e poi risveglia il processo	La stessa system call ritorna con il risultato
Sincrono non bloccante	Continua l'esecuzione	Immediatamente	La chiamata restituisce i dati disponibili oppure EAGAIN	Il processo deve controllare e riprovare, eventualmente usando select()	Ogni nuova chiamata restituisce ciò che è disponibile in quel momento
Asincrono	Continua l'esecuzione	Immediatamente, prima del completamento	Il kernel mantiene attiva e completa la richiesta	Il kernel completa l'operazione in background	Il processo viene notificato tramite segnale, evento o callback

SCHEDULING

Ogni dispositivo può avere una coda di richieste. Il **kernel** decide quale eseguire per prima.

Gli obiettivi possono essere:

- migliorare le prestazioni;
- garantire **fairness**, evitando che un processo monopolizzi il dispositivo;
- offrire **Quality of Service**, rispettando priorità o tempi richiesti.

Trabocchetto: l'ordine più efficiente non è necessariamente il più equo.

Buffering

Un **buffer** è **memoria temporanea** collocata tra chi produce e chi consuma i dati.

Serve per tre ragioni precise:

1. **Differenza di velocità:** il dispositivo produce dati più lentamente o più velocemente del processo.
2. **Differenza nella dimensione dei trasferimenti:** un componente lavora un byte alla volta, un altro a blocchi.

3. **Copy semantics:** il kernel conserva una **copia stabile dei dati**.

Esempio di copy semantics: il processo chiama write(buffer) e subito dopo modifica buffer. La scrittura deve utilizzare i dati forniti al momento della chiamata, non quelli modificati successivamente.

Trabocchetto: il buffering non serve soltanto ad aumentare la velocità; serve soprattutto a **disaccoppiare produttore e consumatore**.

Double buffering

Si utilizzano **due buffer**:

- **mentre il dispositivo riempie il primo;**
- **il sistema elabora il secondo.**

Poi i ruoli si scambiano. In questo modo **trasferimento ed elaborazione possono sovrapporsi**.

Nella slide si considerano spesso una copia nel **kernel** e una nello **spazio utente**, eventualmente di dimensioni diverse. Però l'idea fondamentale è l'alternanza tra due buffer: uno viene usato, mentre l'altro viene riempito o elaborato.

Copy-on-write

Con il **copy-on-write**, due soggetti **condividono inizialmente gli stessi dati**. La copia viene creata solo quando uno tenta di modificarli.

Quindi:

- non elimina necessariamente la copia;
- la **rimanda** finché non avviene una scrittura;
- se nessuno modifica i dati, la copia può essere evitata del tutto.

CH 13

FILE = spazio di indirizzamento logico continuo. Ha vari ATTRIBUTI: nome, identificatore (identifica il file all'interno del File System), tipo, location (puntatore), protezione (controlla accessi: lettura, scrittura, esecuzione), tempo, data, l'identificatore dello user. Le informazioni (metadati) sono contenuti in una directory structure allocata sul disco. **Il file è un abstract data type** (il programma lo utilizza attraverso un insieme di operazioni ben definite, senza conoscere la sua implementazione fisica nel file system). I dettagli sono nascosti dal SO. Non è necessario che il programma conosca come il sistema operativo memorizza fisicamente il file sul disco. Operazioni offerte: create, open, read, write, seek (modifica la posizione corrente), close, delete, truncate.

Un file in FILE SYSTEM può essere organizzato in vari modi:

- **SEQUENZA DI BYTE:** Il SO conosce la posizione dei byte, ma non il loro significato. E' il programma a saperli interpretare. Accetta sia **ACCESSO SEQUENZIALE** (leggere i byte in

ordine uno dopo l'altro), che **ACCESSO DIRETTO** (andare ad un determinato offset ad es con operazioni come lseek() e leggere direttamente da quel punto).

- **STRUTTURA A RECORD**: file suddiviso in unità dati logiche chiamate RECORD
 - **LINEE**→ ogni record corrisponde ad una riga che termina con \n. Si ha sia **ACCESSO SEQUENZIALE** (si leggono le righe una dopo l'altra), ma **NON ACCESSO DIRETTO**: più complicato perché le righe possono avere lunghezze differenti, quindi o si utilizza un **INDICE** che memorizza l'offset di ogni riga, oppure si leggono tutte le righe precedenti. A partire dal numero di riga non è possibile ricavare direttamente la sua posizione.
 - **RECORD A LUNGHEZZA FISSA**: tutti i record hanno la stessa dimensione. Sia **ACCESSO SEQUENZIALE** che **DIRETTO (particolarmente efficiente perché la posizione del record i-esimo si calcola con $\text{offset}(i) = i * \text{dimensione_record}$)**
 - **RECORD A LUNGHEZZA VARIABILE** : hanno dimensioni diverse. Si **ACCESSO SEQUENZIALE**, più complicato **ACCESSO DIRETTO**.
- **STRUTTURA COMPLESSA**: può essere un **documento formattato** (contiene dati + metadati) che ha sia **accesso sequenziale, quello diretto dipende dal formato** (se il file contiene indici o tabelle interne può raggiungere direttamente una determinata sezione), oppure **relocatable load file** (file oggetto con codice macchina, info di rilocalizzazione...), ha sia accesso diretto che sequenziale.

GESTIONE FILE APERTI

Quando un processo esegue open() il sistema operativo non restituisce direttamente un puntatore ai dati sul disco, ma crea una struttura in memoria (in KERNEL) chiamata **OPEN-FILE TABLE** che ha una entry per ogni file aperto. Restituisce al processo un **FILE DESCRIPTOR** che è un intero che identifica il file aperto. Ogni entry contiene:

puntatore al file nel file system (vnode), offset corrente, modalità di apertura, contatore dei riferimenti. Il fd identifica una entry della tabella.

Con il contatore di riferimenti si intende il **FILE-OPEN COUNT** che indica quanti file descriptor fanno riferimento alla stessa open-file entry.

Se ci sono + processi che accedono contemporaneamente allo stesso file si cerca di evitare dei dati incoerenti o stati inconsistenti usando il **FILE LOCKING**. Abbiamo **SHARED LOCK** (tipo reader lock, più processi possono acquisirlo contemporaneamente perché stanno leggendo il file senza modificarlo), **EXCLUSIVE LOCK** (tipo writer lock, solo un processo può averlo perché modifica il file). Shared lock ed exclusive lock sono mutuamente esclusivi.

Il lock può essere anche **MANDATORY** (se imposto dal sistema operativo, non può essere ignorato dal processo), oppure **ADVISORY** (SO/kernel impone il lock, ma sta ai programmi rispettarlo volontariamente, altrimenti può comunque accedere al file).

METODI DI ACCESSO

Stabiliscono in quale ordine un programma può leggere o scrivere i dati di un file.

- **SEQUENZIALE**→ dati elaborati in ordine partendo dalla posizione corrente. C'è un puntatore che indica il prossimo elemento da leggere/scrivere : **current position**. Le operazioni principali sono: **read next** (legge l'elemento corrente e sposta il puntatore al

successivo), **write next** (simile ma per la scrittura), **reset/rewind**: riporta il puntatore all'inizio del file.

A | B | C | D | E



posizione corrente

Per leggere E bisogna leggere prima A,B,C,D.

- DIRETTO (random access): file considerato come un insieme di record/blocchi logici di dimensione fissa solitamente e numerati. E' possibile accedere direttamente ad un blocco specificandone il numero. Operazioni: read(n), write(n), position to n (per spostare il puntatore al blocco n). n è il **relative block number (indica la posizione del blocco rispetto all'inizio del file, non la sua posizione fisica sul disco).**

Con i record di dimensione fissa possiamo calcolarne la posizione:

$$\text{offset} = n * \text{dimensione record}$$

L'accesso diretto consente anche di leggere un file in ordine sequenziale.

- INDICIZZATO: cerchiamo un record attraverso una CHIAVE, un INDICE.

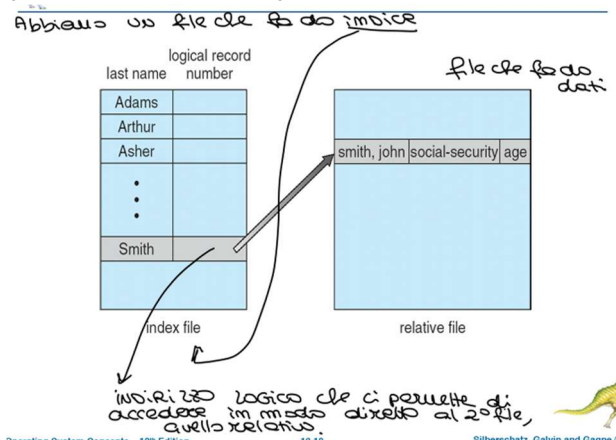
Chiave → posizione del record.

La posizione punta direttamente al record nel file di dati.

- FILE INDICE E FILE RELATIVO: abbiamo 2 file:

- INDEX FILE → contiene la chiave di ricerca e il numero logico del record corrispondente (Adams → 12).

- RELATIVE FILE → contiene i dati completi organizzati in record logici numerati (Adams → 12 → dati...).



- APPROCCIO MISTO: (ISAM) = Index Sequential-Access method combina sia approccio sequenziale che diretto. Il file dei dati è ordinato secondo una determinata chiave e può essere letto sia sequenzialmente oppure attraverso un INDICE per accedere ad un record.

ORGANIZZAZIONE DEL DISCO

DIRECTORY = struttura che contiene info sui file, associa nome del file alla struttura utilizzata dal file system per identificarlo e localizzarlo.

Sia le directory che i file sono memorizzati sul disco. Si parla in particolare di un'organizzazione logica detta STRUTTURA DELLE DIRECTORY (struttura gerarchica a + livelli).

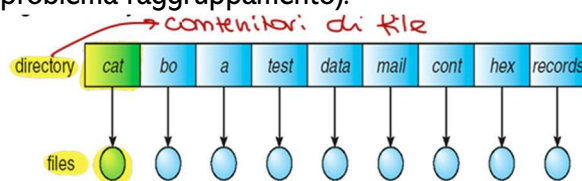
Il disco può essere suddiviso in una o + PARTIZIONI (porzione logica del disco, trattata come una unità separata). Una partizione può essere usata in 2 modi:

- Raw → non contiene un file system, ci sono i blocchi grezzi a cui un programma o il SO possono accedervi direttamente.
- Formattata → contiene un file system, strutture necessarie per organizzare i file, le directory, info su file...

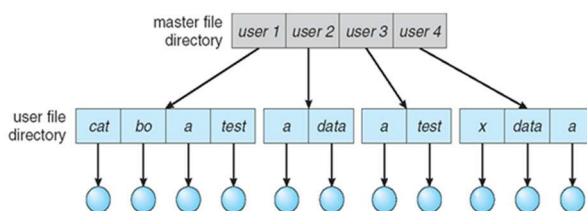
VOLUME = unità logica di memorizzazione che può contenere un file system. Di solito una partizione formattata coincide con il volume. Ma in realtà il volume può contenere anche + partizioni o estendersi su più dischi (dispositivi fisici).

Organizzazione delle directory

- DIRECTORY A SINGOLO LIVELLO: una sola directory contiene tutti i file, per tutti gli utenti. I nomi devono essere unici. E' una struttura semplice, ma adatta solo a sistemi piccoli. Inoltre non si può usare lo stesso nome per file diversi e non esistono sottodirectory (problema raggruppamento).



- TWO-LEVEL DIRECTORY: ogni utente ha una directory personale chiamata User File Directory (UFD). C'è una directory principale chiamata Master File Directory (MFD) che contiene un riferimento alla directory di ogni utente.

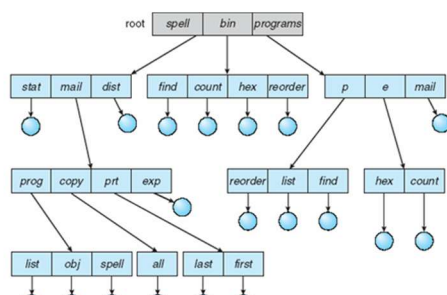


In questo modo utenti diversi possono creare file con lo stesso nome perché tanto si trovano su directory diverse: /user1/test, /user2/test. Ricerca + efficiente (solo nella

directory dell'interessato). File identificato da pathname (percorso costituito da nome utente + nome file). PROBLEMA: l'utente non può creare sottodirectory (no grouping).

- TREE STRUCTURED DIRECTORY: introduce + livelli di directory e sottodirectory. Si parte dalla root directory (/) e da lì partono tutte le directory. I file sono le foglie, le directory sono i nodi intermedi, ogni file/directory (tranne la root) ha una directory padre. VANTAGGI: raggruppamento (file organizzati per utente, tipo...), nomi ripetuti, ricerca + efficiente (conosco il percorso, si limita alla directory interessata).

PATHNAME = posizione del file in albero. Può essere assoluto (si parte dalla root: /spell/mail/prog/list, non dipende dalla directory corrente), oppure relativo (/spell/mail/prog, parte dalla current directory).

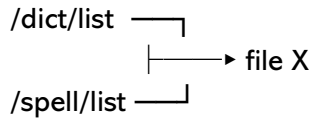


- ACYCLIC GRAPH DIRECTORIES

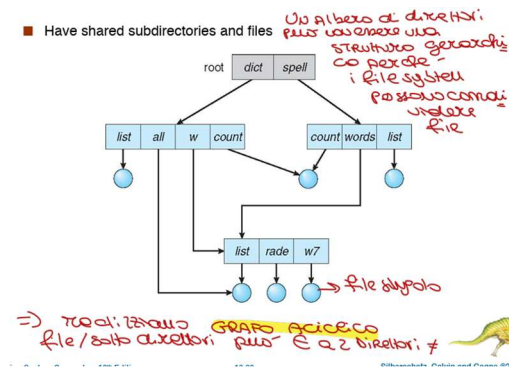
Con la struttura ad albero ogni file/ directory ha un solo padre, invece con il GRAFO riusciamo a condividere lo stesso oggetto tra utente / directory diversi.

Nel grafo aciclico + utenti condividono uno stesso file / sottodirectory. Quindi un nodo può avere più padri. Il file non viene copiato, ma è raggiungibile da directory diverse. Per condividere un file, dobbiamo aggiungere una nuova entry nella directory chiamata **LINK**. Questa contiene un riferimento ad un file già esistente. Lo stesso file è raggiungibile con nomi/percorsi diversi (diversi pathname = aliasing).

PROBLEMA : dangling pointer:



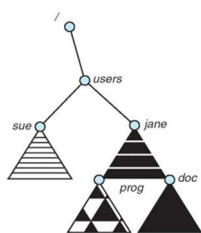
Se cancellassimo /dict/list, questo provocherebbe anche la cancellazione di file x e quindi il link /spell/list punterebbe ad un oggetto inesistente. Si possono usare per risolvere questo problema: contatore di riferimenti (elimino il file solo quando count = 0, soluzione + usata), daisy chain di backpointer (catena di link, non dobbiamo riservare memoria in anticipo), backpointer (contiene info sui link che puntano ad uno stesso file ma dobbiamo riservare un'area di memoria fissa in anticipo).



- **GRAFO GENERALE:** consente anche collegamenti tra directory che possono produrre cicli. **PROBLEMA:** potrebbe non terminare mai (Due directory isolate potrebbero continuare a riferirsi reciprocamente e mantenere il contatore maggiore di zero.) Per risolvere:
 - si consente la condivisione di file ma non nelle sottodirectory (i file non contengono altre directory, quindi evitiamo ciclo, è la + semplice per costruzione)
 - quando viene creato un nuovo link si esegue un algoritmo per rilevare eventuali cicli. Il link viene accettato solo se non li crea (preciso ma costoso).
 - Garbage collection → periodicamente il sistema individua degli oggetti non + raggiungibili dalla root e libera lo spazio, elimina gruppi di directory che formano un ciclo ma non sono + accessibili.

MONTAGGIO DI UN FILE SYSTEM

Prima di poter accedere a un file system, il sistema operativo deve eseguire il **mount**, cioè collegarlo all'albero delle directory già esistente. Un file system non montato esiste sul disco, ma non è ancora raggiungibile tramite un normale percorso. Il **mount point** è la directory dell'albero esistente alla quale viene collegato il nuovo file system. Il mount NON COPIA i file.



tipo in questo esempio abbiamo collegato il file system alla directory /users.

Se la directory usata come mount point conteneva già dei file, questi vengono temporaneamente nascosti dal file system montato. Tornano visibili dopo l'**unmount**.

Condivisione di file

Nei sistemi multiutente è utile permettere a più utenti di accedere agli stessi file. La condivisione deve però essere accompagnata da un sistema di **protezione**, per stabilire chi può compiere determinate operazioni.

Ogni utente è identificato tramite:

- **UID – User ID**: identifica un singolo utente;
- **GID – Group ID**: identifica un gruppo di utenti.

Ogni file o directory possiede:

- un **owner**, cioè l'utente proprietario;
- un **group**, cioè il gruppo associato.

In questo modo il sistema può assegnare diritti differenti al proprietario, al gruppo e agli altri utenti.

Un file può essere condiviso anche tra computer collegati in rete (sistemi remoti). Esistono 3 modalità: FTP (utente carica/scarica manualmente il file e poi lavora su una copia locale), accesso automatico tramite file system distribuito (file system remoto viene montato e appare come se fosse locale), accesso da web (utente accede a file con browser che gestisce la comunicazione).

In questi casi si usano spesso modelli client-server con protocolli come NFS, CIFS/SMB per condividere file.

La SEMANTICA DELLA CONDIVISIONE stabilisce cosa accade quando più utenti o processi accedono contemporaneamente allo stesso file.

Chi crea il file deve decidere cosa possono fare gli altri su quel file e CHI può fare (PROTEZIONE).

ACCESS CONTROL LIST = esplicitiamo per un file chi e cosa può fare.

3 bit: owner access, group access, public access. Es 761 (111 110 001 = RWX, RW, X).

Ch 11

La memoria di massa (secondaria, disco) conserva i dati e i programmi in modo permanente (anche in assenza di alimentazione). Principalmente abbiamo come tecnologie:

- **HDD** (Hard Disk Drive): disco magnetico con componenti meccaniche in movimento
- **NVM** (Non volatile Memory): tipo SSD

HDD

E' costituito da piatti magnetici concentrici che ruotano attorno ad un asse centrale detto spindle. La superficie del piatto è suddivisa in TRACCE (circonferenze concentriche), a loro volta suddivise in SETTORI (porzioni di una traccia in cui vengono memorizzati i dati).

Capacità = numero di settori × byte per settore

Le tracce poste alla stessa distanza dal centro sui diversi piatti formano CILINDRO = insieme delle tracce concentriche equidistanti dal centro.

La velocità di rotazione è espressa in **RPM** (rotazioni per minuto). Alcuni dischi sono rimovibili e se la testina entra in contatto con il piatto può verificarsi un head crash che danneggia la superficie magnetica e causa la perdita di dati.

Per le performance:

Taccesso medio (access latency) = Tseek medio+ T latenza rotazionale → rappresenta il tempo necessario per posizionarsi sul dato

T I/O=Taccesso+Ttrasferimento+Toverhead del controller

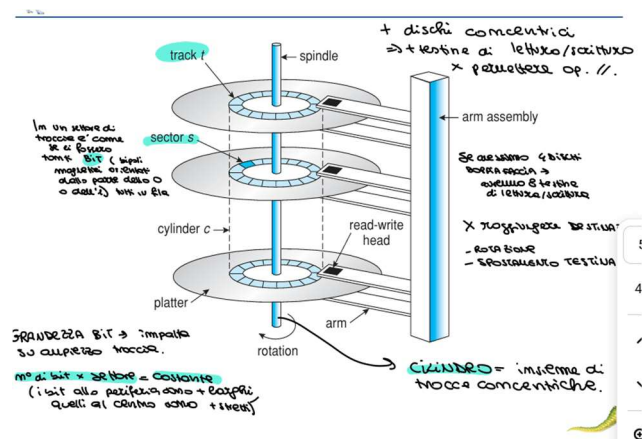
Transfer rate = velocità con cui i dati vengono trasferiti tra disco e computer

Positioning time / random-access time = tempo necessario per raggiungere fisicamente i dati

Seek time = tempo impiegato dal braccio per raggiungere la traccia desiderata

Rotational Latency = attesa affinché il settore desiderato ruoti sotto la testina.

La dimensione del settore non indica il numero di bit/byte memorizzati.



Le testine sono collegate allo stesso arm assembly e quindi si spostano insieme sul medesimo cilindro, sarà poi il controller a selezionare elettronicamente quale testina usare. Ogni superficie ha una testina.

Per accedere ad un dato sono necessarie 2 operazioni:

- Spostamento del braccio verso la TRACCIA corretta
- Rotazione del disco fino a mettere il SETTORE sotto la testina.

Per questo l'accesso casuale in un HDD è lento. SVANTAGGIO: il tempo di trasferimento è quasi trascurabile rispetto al tempo meccanico necessario per posizionare la testina.

Per gli esercizi:

$$T_{\text{latenza media}} = \frac{T_{\text{rotazione completa}}}{2} \quad T_{\text{rotazione}} = \frac{60}{\text{RPM}} \text{ s} \quad T_{\text{trasferimento}} = \frac{\text{dimensione dei dati}}{\text{transfer rate}}$$

Caso migliore:

$$T_{\text{rot.min}} = 0$$

Caso peggiore:

$$T_{\text{rot.max}} = T_{\text{rotazione}}$$

$$R_{\text{effettivo}} = \frac{\text{dimensione trasferita}}{T_{\text{I/O}}}$$

6. Tempo totale di I/O

$$T_{\text{I/O}} = T_{\text{seek}} + T_{\text{rot.media}} + T_{\text{trasferimento}} + T_{\text{controller}}$$

NVM

Non volatile memory → conservano i dati anche senza alimentazione. Si parla principalmente di SSD che a differenza degli HDD: non possiedono testine, bracci o piatti in movimento (no parti meccaniche), non hanno **seek time** e **latenza rotazionale**. Sono + veloci e resistenti agli urti, consumano meno energia. Hanno un costo maggiore e capacità inferiore. Hanno un numero limitato di scritture (serve gestione specifica).

Questo tipo di memorie NAND sono organizzate in BLOCCHI → PAGINE. La **pagina** è l'**unità minima di lettura/scrittura** (tipo **settore** di un HDD). Il **blocco** (formato da + pagine) è l'**unità minima di cancellazione**.

Una pagina può essere letta e scritta, ma non può essere sovrascritta direttamente. Il controller, per modificarla, deve scrivere la nuova versione dei dati in una pagina libera, segnare la vecchia pagina come "non valida" e cancellare successivamente l'intero blocco che contiene le pagine

$$\text{lettura/scrittura} = \text{pagina}$$

non valide.

$$\text{cancellazione} = \text{blocco}$$

La cancellazione di un blocco è relativamente lenta e può essere eseguita un numero limitato di volte. Infatti la durata di un SSD può essere espressa tramite DWPD (Drive Writes Per Day = numero di volte in cui l'intera capacità del disco può essere scritta ogni giorno durante il periodo di garanzia.) Una scrittura o riscrittura è sempre preceduta da una cancellazione.

$$\text{Scritture giornaliere} = \text{capacità SSD} \times \text{DWPD}$$

Il DWPD non misura quante singole operazioni di scrittura si possono eseguire, ma il **volume complessivo di dati** scrivibile ogni giorno.

Poiché non è possibile sovrascrivere direttamente una pagina, nel tempo i blocchi contengono una combinazione di:

- **pagine valide**, contenenti la versione corrente dei dati;
- **pagine non valide**, contenenti vecchie versioni che non devono più essere usate;
- **pagine libere**, ancora disponibili per nuove scritture.

Il controller gestisce questa situazione mediante quattro meccanismi.

- FTL = Flash Translation Layer : tabella di traduzione che associa ogni indirizzo logico visto dal sistema operativo alla pagina fisica in cui si trova realmente il dato:

$$\text{blocco logico} \rightarrow \text{pagina fisica}$$

Quando un dato viene aggiornato, il controller lo scrive in una nuova pagina e aggiorna la FTL; la vecchia pagina diventa non valida. Il sistema operativo continua però a usare lo stesso indirizzo logico.

- Garbage Collection: recupera lo spazio occupato dalle pagine non valide: seleziona un blocco contenente pagine valide e non valide; copia le pagine ancora valide in un altro blocco; cancella completamente il vecchio blocco; rende nuovamente disponibili tutte le sue pagine. Questo processo può produrre scritture interne aggiuntive.

Overprovisioning: parte della memoria fisica non mostrata all'utente. Viene utilizzata dal controller come spazio di lavoro per: spostare le pagine valide; eseguire la garbage collection; sostituire blocchi usurati; mantenere buone prestazioni anche quando l'SSD è quasi pieno.

Wear Leveling: distribuisce uniformemente le scritture e le cancellazioni tra i blocchi. Senza questo meccanismo, i blocchi utilizzati più frequentemente si usurerebbero molto prima degli altri.

RAM DRIVE/RAM DISK (DRAM) = parte della RAM usata come disco, sulla quale può essere creato un file system. Non è proprio una memoria secondaria essendo volatile e quindi allo spegnimento si perdono i dati. Viene comunque utilizzata come memoria di massa temporanea perché + veloce di HDD e SSD e utile per: scambio rapido di grandi quantità di dati, file temporanei, dati intermedi, operazioni che richiedono molte operazioni veloci.

Mentre le cache/buffer sono gestiti automaticamente dai programmi/SO/HW, il RAM DRIVE è creato direttamente dall'utente e usato come un disco.

Il SO vede il disco come una grande array monodimensionale di blocchi logici: B0, B1, B2... Bn. Il blocco logico è la + piccola unità di trasferimento indirizzabile dal sistema, invece la formattazione di basso livello prevede come supporto fisico i settori. Prima si parte dal primo settore della prima traccia, poi si scorre attraverso i settori di quella stessa traccia, si attraversano le superfici dello stesso cilindro e si passa ai cilindri successivi.

SCHEDULING DELL'HDD

Lo scheduling stabilisce l'ordine in cui servire le richieste di I/O in attesa. **L'OBJ principale è: minimizzare il movimento della testina per ridurre il tempo di accesso.** Il tempo di seek infatti è proporzionale alla distanza percorsa dalla testina: + cilindri attraversa a testina e > è il seek time. Se riduciamo il movimento totale, diminuisce anche il tempo medio di accesso e aumenta la banda del disco.

La disk bandwidth è:

$$\text{Banda} = \frac{\text{byte complessivamente trasferiti}}{\text{tempo tra la prima richiesta e la fine dell'ultimo trasferimento}}$$

Gli algoritmi di scheduling hanno senso solo quando ci sono + richieste in attesa. Le richieste possono venire da SO, processi utente, processi di sistema e vengono riordinate dal controller interno del dispositivo. Per ogni disco / dispositivo abbiamo una coda di richieste. Se il disco è libero la richiesta viene servita subito, altrimenti inserita in coda. Ogni richiesta specifica l'operazione di lettura/scrittura, l'indirizzo logico del disco (LBA), indirizzo memoria centrale, numero di settori da trasferire.

Consideriamo i cilindri numerati da 0 a 199, con:

coda = 98, 183, 37, 122, 14, 124, 65, 67

La testina parte dal cilindro:

53

Il movimento tra due cilindri a e b si calcola con:

$$d = |b - a|$$

Il movimento totale è la somma di tutti gli spostamenti:

$$D_{\text{totale}} = \sum_{i=1}^n |p_i - p_{i-1}|$$

dove p_0 è la posizione iniziale della testina.

E' il CONTROLLER del dispositivo di IO che usa un algoritmo per ottimizzare le richieste di I/O che arrivano da SO e vengono memorizzate in un buffer di I/O. Ognuna punta ad un settore diverso e noi dobbiamo stabilire l'ordine con cui raggiungiamo i vari settori.

- FCFS: First Come First Served → serve le richieste esattamente nell'ordine di arrivo. E' semplice, fairness ma troppi spostamenti inutili con la testina.

Ordine:

53 → 98 → 183 → 37 → 122 → 14 → 124 → 65 → 67

Calcolo:

$$D = |98 - 53| + |183 - 98| + |37 - 183| + |122 - 37| + |14 - 122| + |124 - 14| + |65 - 124| + |67 - 65|$$

$$D = 45 + 85 + 146 + 85 + 108 + 110 + 59 + 2$$

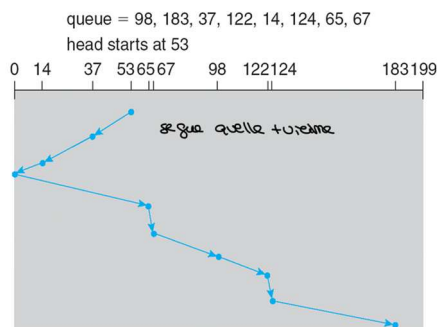
$$D_{\text{FCFS}} = 640 \text{ cilindri}$$

Valutazione

- semplice e senza starvation;
- rispetta l'ordine di arrivo;
- poco efficiente: la testina cambia continuamente direzione;
- adatto soprattutto quando il carico è basso.

Il passaggio tra cilindri influenza il seek time; il passaggio tra settori della stessa traccia riguarda invece principalmente la latenza rotazionale.

- SCAN = Algoritmo dell'ascensore. Sposta la testina in una direzione servendo tutte le richieste incontrate, arrivato ad un'estremità inverte il movimento.



La slide indica 208 cilindri, ma questo risultato si ottiene con LOOK, non con SCAN puro.

LOOK inverte la direzione all'ultima richiesta presente, cioè 14, senza arrivare fisicamente a 0:

53 → 37 → 14 → 65 → 67 → 98 → 122 → 124 → 183

$$D = (53 - 14) + (183 - 14)$$

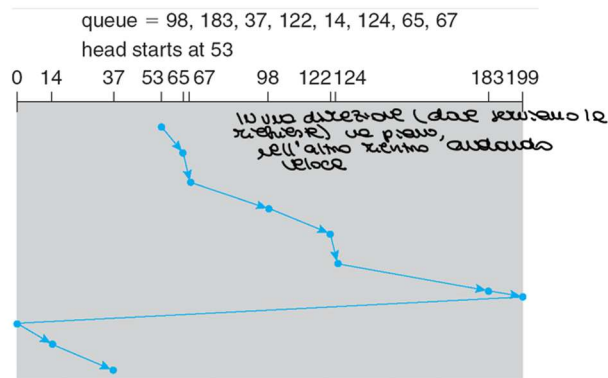
$$D_{\text{LOOK}} = 39 + 169 = 208 \text{ cilindri}$$

Quindi:

- se arriva fino a 0: SCAN = 236;

SCAN: riduce il rischio di starvation, quindi è più efficiente di FCFS, ma dopo l'inversione le richieste vicine all'estremità appena lasciata possono attendere + a lungo.

- C-SCAN = Circular SCAN: serve le richieste in una sola direzione. Quando raggiunge un'estremità ritorna direttamente all'altra senza servire richieste durante il ritorno.



53 → 65 → 67 → 98 → 122 → 124 → 183 → 199

Poi ritorna direttamente:

199 → 0

e riprende:

0 → 14 → 37

Il disco viene quindi trattato come una lista circolare:

0, 1, 2, ..., 199, 0, 1, ...

Il movimento totale è:

$$D = (199 - 53) + (199 - 0) + (37 - 0)$$

$$D = 146 + 199 + 37$$

$D_{C-SCAN} = 382 \text{ cilindri}$

Produce tempi di attesa più uniformi rispetto a SCAN: tutte le richieste vengono servite nella stessa direzione e nessuna riceve un vantaggio dovuto al viaggio di ritorno.

Un movimento totale minore indica generalmente un seek time complessivo minore, ma non basta da solo: bisogna considerare anche equità, tempo medio di attesa e rischio di starvation.

- SSTF = Shortest Seek Time First → seleziona ogni volta la richiesta + vicina alla posizione corrente. Offre buone prestazioni e riduce il movimento, ma può causare **starvation**: una richiesta lontana potrebbe essere rimandata continuamente se continuano ad arrivare richieste più vicine. SCAN e C-SCAN si comportano meglio con carichi elevati perché servono le richieste secondo un movimento ordinato della testina e limitano la starvation. SCAN e C-SCAN si usano con HDD perché la sua testina deve spostarsi tra cilindri di versi e quindi dobbiamo ottimizzare il tempo medio di seek. C-SCAN offre tempi di attesa + uniformi perché tratta i cilindri come una sequenza circolare.

Linux usa deadline scheduler: mantiene code separate per letture e scritture. Assegna la priorità alle letture perché spesso un processo rimane bloccato in attesa di leggere i dati. Assegna una scadenza alle richieste. Mantiene code ordinate per LBA e code FCFS per controllare da quanto tempo aspettano le richieste.

SCHEDULING SSD / NVM (di solito no scheduling)

Negli SSD non esistono testine, seek time o latenza rotazionale. Di conseguenza, riordinare le richieste in base alla distanza tra LBA è molto meno importante. Può essere usato **NOOP**, uno scheduler minimale che non esegue un vero riordinamento meccanico,

ma può unire richieste rivolte a LBA adiacenti. NOOP significa NO OPERATION: il SO non cerca di riordinare le richieste per ridurre gli spostamenti della testina. Le richieste vengono gestite in FIFO. Si usa con NVM / SSD perché non hanno testina meccanica, latenza rotazionale, tempo di seek e hanno tempo di accesso indipendente dalla posizione fisica dei dati.

Le differenze principali sono:

- gli HDD sono efficienti soprattutto negli accessi sequenziali;
- gli SSD sono molto più efficienti negli accessi casuali;
- il throughput sequenziale può talvolta essere simile;
- gli SSD possiedono un numero di IOPS molto maggiore.

Gli IOPS indicano quante operazioni di I/O possono essere completate ogni secondo:

$$IOPS = \frac{\text{numero di operazioni}}{\text{tempo in secondi}}$$

Le prestazioni degli SSD possono però diminuire a causa della **write amplification**: una singola scrittura logica può provocare più operazioni fisiche interne, dovute allo spostamento delle pagine valide e alla garbage collection.

$$WA = \frac{\text{dati fisicamente scritti nella NAND}}{\text{dati scritti dal sistema}}$$

Se $WA > 1$, l'SSD scrive fisicamente più dati di quelli richiesti dal sistema, con perdita di prestazioni e maggiore usura.

Rilevazione degli errori

Con la rilevazione si va a verificare se i dati sono stati alterati (es cambiamento accidentale di un bit da 1 → 0 o viceversa). Se viene rilevato un errore in questo caso il sistema può interrompere l'operazione oppure richiedere nuovamente i dati.

- BIT DI PARITÀ → bit aggiuntivo calcolato sui dati. Permette di rilevare ogni errore su un singolo bit ma non indica quale bit sia errato e non si può correggere. Errori multipli talvolta non sono rilevati. Con parità pari il numero complessivo di 1 deve essere pari, con dispari deve essere dispari.
- CHECKSUM = calcola un valore sintetico a lunghezza fissa dai dati.
checksum ricevuto ≠ checksum calcolato ⇒ errore
- CRC = Cycle Redundancy Check → Individua sequenze di errori su + bit. Usa divisioni polinomiali in aritmetica modulare.
- ECC = Error Correcting Code: aggiunge ridondanza per rilevare errori, e **correggerli**. Gli **errori soft**, cioè alterazioni occasionali dei bit, possono spesso essere corretti. Gli **errori hard**, provocati da un guasto fisico permanente, possono essere rilevati ma non sempre corretti.

Per poter contenere un file, un disco deve:

- Ricevere **low-level formatting**: divider fisicamente il disco in settori leggibili e scrivibili dal controller del dispositivo di I/O. Definisce l'organizzazione fisica del supporto. Ogni settore ha: un'intestazione con info di controllo, dati, ECC per rilevare e correggere errori.
- Essere **partizionato**: dividere il disco in unità logiche, ognuna trattata da SO come un disco indipendente. Queste partizioni possono contenere: SO, file system, dati grezzi (no file system), spazio di swap
- Essere **formattato logicamente**: all'interno di una partizione si crea un file system (definendo strutture necessarie per gestire file + directory). Molti file system raggruppano i blocchi fisici in CLUSTER (unità di allocazione dei file).

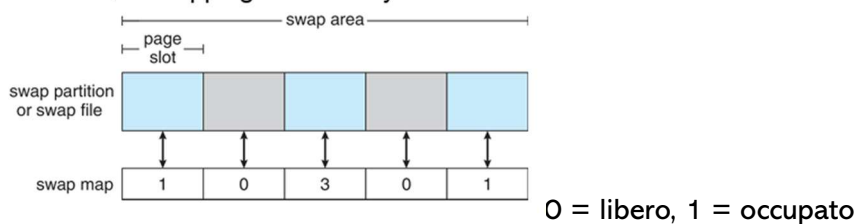
La **root partition** contiene il file system principale del sistema operativo. Viene montata durante il boot; le altre partizioni possono essere montate automaticamente oppure manualmente. Il **boot block** contiene o individua il codice necessario ad avviare il sistema. Può contenere un piccolo boot loader, che conosce la posizione del kernel oppure sa come leggerlo dal file system.

Se non c'è il file system abbiamo una "raw disk access": l'applicazione accede direttamente ai blocchi di una partizione. E' l'applicazione a scegliere come organizzare/recuperare le informazioni, dove memorizzare i dati e quali blocchi sono liberi.

All'accensione il processore esegue automaticamente il codice contenuto nel firmware (BIOS) e viene individuato il dispositivo dal quale effettuare l'avvio. Legge il primo settore del disco chiamato Master Boot Record (MBR) che contiene BOOT CODE : piccolo codice eseguibile che avvia, PARTITION TABLE: contiene l'elenco delle partizioni del disco indicando quale è avviabile. Grazie a questa tabella, l'MBR individua la boot partition (quella da cui deve essere avviato il sistema) che contiene i settori chiamati BOOT BLOCK. In BOOT BLOCK si trova il BOOTSTRAP LOADER (un programma completo che comprende la struttura del file system, individua il file del kernel, carica il kernel dal disco alla RAM e trasferisce al kernel il controllo della CPU). Poi viene eseguito il kernel.

SWAP SPACE

= zona di memoria secondaria usata come estensione temporanea della RAM, nel caso in cui questa non sia + sufficiente (DRAM). In questo modo il SO può trasferire sul disco un'intero processo(swapping) o soltanto alcune pag (paging). Quando una pag che sta nello swap serve nuovamente, viene riportata in RAM.



3 modalità di accesso al disco:

- Host-attached: disco fisicamente collegato al server/computer (disco USB, SSD...). Server vede un disco locale e gestisce direttamente il file system.
- NAS: computer che contiene dischi e proprio file system. Il server accede attraverso la rete LAN/WAN a cartelle /file condivisi.
- Cloud: lo storage si trova generalmente nel data center remoto, a cui si accede attraverso Internet / API. I dati vengono serviti come un servizio remoto via Internet.

COMPONENTI DI UN'INFRASTRUTTURA PROFESSIONALE:

- Storage array: contiene numerosi HDD /SDD, controller, memoria cache. Il controller combina / gestisce i dischi tramite RAID.
- SAN = rete specializzata che collega uno o + server agli storage array. Il server riceve i blocchi e vede lo spazio remoto come un disco locale.
- FC = fibra channel, tecnologia di comunicazione veloce e affidabile per traffico dello storage.

RAID

RAID = Redundant Array of Independent Disks (inexpensive disks). Si tratta di + dischi che vengono visti dal SO come un'unica unità logica.

VANTAGGI: aumentano le prestazioni (+ dischi in parallelo), l'affidabilità (info ridondanti).

2 tecniche:

- STRIPING: distribuzione dei dati
- RIDONDANZA: duplicazione /parità per recuperare dati persi.

Striping

Dati suddivisi in CHUNCK/STRIPE UNIT e distribuiti su + dischi. Aumentano le prestazioni perché aumenta il parallelismo e quindi il throughput. No affidabilità. Posso lavorare contemporaneamente sui blocchi distribuiti tra i vari dischi.

Ridondanza

Memorizza info aggiuntive che consentono di ricostruire un dato in caso di guasto. Si ottiene con:

- MIRRORING: si conserva una copia COMPLETA
- PARITA': si conserva un valore calcolato sui dati.

Aumenta l'affidabilità.

$$\text{capacità utile} < \text{capacità fisica totale}$$

AFFIDABILITA': MTTF, MTTR, MTDDL

MTTF = Mean Time To Failure = tempo medio prima del guasto di un componente:

$$MTTF = \text{tempo medio fino al guasto}$$

MTTR = Mean Time To Repair = tempo medio per riparazione e ricostruzione, per sostituire il disco guasto e ricostruire su quello i nuovi dati ridondanti. Il RAID continua a funzionare.

MTDDL = Mean Time To Data Loss = tempo medio fino alla perdita dei dati. Un guasto non implica una perdita dei dati necessariamente. Ad es in RAID 1 se si guasta il primo disco, non perdiamo i dati perché esiste la copia, se si guasta anche la copia allora perdiamo anche i dati.

ESEMPIO: supponiamo di avere 2 dischi in mirroring. Hanno MTTF = 100000 ore e MTTR = 10 ore . Quindi ciascun disco considerato separatamente si guasta mediamente dopo 100 000 ore. Ogni disco ha un tasso medio di guasto = $1/100\,000$. Considerandoli entrambi, i tassi di guasto si sommano:

$$\lambda_{\text{coppia}} = \lambda_A + \lambda_B = \frac{1}{100\,000} + \frac{1}{100\,000} = \frac{2}{100\,000}$$

Il tempo medio fino al primo guasto è l'inverso del tasso complessivo:

$$MTTF_{\text{primo guasto}} = \frac{1}{\lambda_{\text{coppia}}} = \frac{1}{2/100\,000} = \frac{100\,000}{2} = 50\,000 \text{ ore}$$

Quindi il primo guasto arriva mediamente prima perché ci sono 2 possibili dischi che possono rompersi. Non significa che ora ogni disco ha un MTTF = 50 000

Dopo il primo guasto rimane solo un disco con i dati. Il sistema rimane vulnerabile per $MTTR = 10h$ ma continua a funzionare. Il disco superstite ha un $MTTF = 100\,000$ ore e la

$$P_{\text{secondo guasto}} \approx \frac{MTTR}{MTTF}$$

$$= \frac{10}{100\,000} = 10^{-4}$$

probabilità che si guasti nelle successive 10 h è

Quindi, dopo ogni primo guasto, la probabilità di perdere anche la copia durante la riparazione è circa:

$$10^{-4} = 0,0001 = 0,01\%$$

Un evento di rischio si presenta mediamente ogni 50 000 ore, ma solo 1/100 000 provoca l'effettiva perdita di dati.

$$MTTDL \approx \frac{MTTF_{\text{primo guasto}}}{P_{\text{secondo guasto}}} = \frac{MTTF/2}{MTTR/MTTF}$$

$$MTTDL \approx \frac{MTTF^2}{2MTTR}$$

$$MTTDL = \frac{(100\,000)^2}{2 \cdot 10} = \frac{10^{10}}{20} = 5 \cdot 10^8 \text{ ore}$$

$$MTTDL = 500\,000\,000 \text{ ore}$$

RAID 0: solo STRIPING. Non aggiunge ridondanza. Non ha ridondanza, quindi non tollera

guasti. CAPACITA' UTILE considerando N dischi:

$$C_{\text{utile}} = N \cdot S$$

- prestazioni elevate grazie al parallelismo;
- capacità utile pari alla somma delle capacità;
- nessuna copia e nessuna parità;
- il guasto di un solo disco compromette l'intero RAID.

RAID 1: MIRRORING. Conserva i dati in 2 copie identiche. Ogni scrittura deve essere eseguita su entrambi i dischi. Una lettura può invece essere servita da una qualsiasi delle due copie. Il RAID 1 può continuare a funzionare dopo un guasto, ma resta **degradato** fino alla ricostruzione della copia.

Disco 1	Disco 2
A	A
B	B
C	C

$$C_{\text{utile}} = \frac{N}{2} \cdot S$$

- ottima affidabilità;
- letture potenzialmente più veloci;
- scritture non raddoppiano necessariamente la velocità, perché entrambe le copie devono essere aggiornate;
- utilizza metà della capacità fisica;
- tollera il guasto di un disco per coppia.

La PARITA' consente di ottenere ridondanza usando meno spazio del mirroring. La parità non contiene una copia leggibile dei dati: contiene l'informazione matematica necessaria per ricostruire **un dato mancante**.

RAID 4: utilizza STRIPING a livello di BLOCCO e un disco dedicato esclusivamente alla PARITA'. Tollera 1 guasto. Ad ogni scrittura deve aggiornare il disco di parità e questo è un collo di bottiglia.

Stripe	Disco 1	Disco 2	Disco 3	Disco parità
1	A	B	C	(P_1)
2	D	E	F	(P_2)

$$C_{\text{utile}} = (N - 1)S$$

RAID 5: Distribuisce i blocchi di parità tra tutti i dischi, risolve il collo di bottiglia di RAID 4. Tollera 1 guasto.

Stripe	Disco 1	Disco 2	Disco 3	Disco 4
1	A	B	C	(P_1)
2	D	E	(P_2)	F
3	G	(P_3)	H	I
4	(P_4)	L	M	N

$$C_{\text{utile}} = (N - 1)S$$

- buone letture;
- maggiore efficienza spaziale del mirroring;
- nessun disco è sempre dedicato alla parità;
- le scritture sono più costose perché bisogna aggiornare dati e parità;
- tollera il guasto di un disco.

RAID 6: Conserva 2 informazioni di parità indipendenti (P e Q), distribuiti tra i vari dischi. Si possono ricostruire i dati anche se si guastano contemporaneamente 2 dischi. I guasti tollerati sono 2. Offre maggiore affidabilità di RAID 5, perde la capacità di 2 dischi, ha scritture +

$$C_{\text{utile}} = (N - 2)S$$

lente perché deve aggiornare 2 parità.

RAID 0+1: si creano 2 gruppi RAID 0, di cui uno è la copia mirror dell'altro. Se si rompe un disco l'intero gruppo RAID 0 che lo contiene è inutilizzabile, però c'è l'altro gruppo.

RAID 1+0: prima si creano coppie di dischi in mirroring e poi i dati vengono distribuiti tra le coppie. Se si rompe un disco si perde soltanto una copia della sua coppia. L'altra coppia resta completamente indipendente. RAID 10 può tollerare più guasti contemporaneamente, purché non si rompano entrambi i dischi della stessa coppia.

Per entrambi dobbiamo avere almeno 4 dischi.

RAID 1 + 0 è generalmente più affidabile di RAID 0 + 1

Entrambi hanno capacità:

$$C_{\text{utile}} = \frac{N}{2} S$$

- **RAID:** ridondanza tra dischi dello stesso array;
- **replicazione:** ridondanza tra array differenti

HOT SPARE = disco di riserva già installato e acceso nello storage array ma non utilizzato normalmente per memorizzare i dati. Se si verifica un guasto su un disco, il controller rileva il guasto, attiva l'hot spare disk, ricostruisce i dati del disco guasto sull'hot spare disk e questo entra a far parte del RAID. I DATI vengono ricostruiti DOPO il guasto. Con un hot spare, la ricostruzione parte automaticamente, riducendo il tempo durante il quale un secondo guasto potrebbe provocare la perdita dei dati. **Riduce MTTR e quindi aumenta MTDL.**

$M = MTTF$, $R = MTTR$, $N = \text{dischi}$, $S = \text{capacità di un disco}$

RAID	Capacità utile	Guasti tollerati	MTDL
RAID 0	NS	0	$\frac{M}{N}$
RAID 1, 2 dischi	S	1	$\frac{M^2}{2R}$
RAID 1, N copie	S	$N - 1$	$\frac{M^N}{N! R^{N-1}}$
RAID 4/5	$(N - 1)S$	1	$\frac{M^2}{N(N - 1)R}$
RAID 6	$(N - 2)S$	2	$\frac{M^3}{N(N - 1)(N - 2)R^2}$
RAID 10	$\frac{NS}{2}$	Uno per coppia	$\frac{M^2}{NR}$

$$MTTF_{\text{primo guasto}} = \frac{M}{N}$$

$$P_{\text{guasto durante rebuild}} \approx \frac{R}{M}$$

$$MTTDL_{\text{anni}} = \frac{MTTDL_{\text{ore}}}{8760}$$

CH 14

FILE = un'unità logica di memorizzazione: per l'utente è una sequenza organizzata di byte contenente informazioni correlate, indipendentemente da dove i dati siano fisicamente collocati. Il **file system** è la componente del sistema operativo che organizza file e directory sulla memoria secondaria. Il suo compito fondamentale è tradurre la visione logica dell'utente nella disposizione fisica sul disco. Nome file → identificatore file → blocchi logici → blocchi fisici.

Il trasferimento di I/O avviene a **blocchi**. Un blocco è formato da uno o più settori:

blocco = k * settori

Il settore è l'unità fisica del dispositivo; il blocco è l'unità normalmente utilizzata dal sistema operativo per effettuare l'I/O.

$$\boxed{\text{Traccia} = \text{insieme di settori}} \quad N_{\text{settori totali}} = N_{\text{tracce totali}} \cdot N_{\text{settori per traccia}}$$

Per OGNI file c'è un FCB (File Control Block) = struttura dati contenente i metadati necessari per conoscere e gestire il file.

FCB contiene:

- identificatore file (numero inode)
- tipo e permessi
- proprietario (ACL)
- dimensione
- data creazione, accesso, modifica
- puntatori ai blocchi contenenti i dati.

FCB = informazioni sul file

blocchi dati = contenuto effettivo del file

Nei sistemi UNIX: abbiamo **l'inode** come FCB. Nelle directory c'è il nome del file e il riferimento all'inode. in NTFS le informazioni analoghe si trovano nella **Master File Table (MFT)**.

nome del file → numero di inode → FCB/inode → blocchi dati

Il File system è suddiviso in vari livelli:

Livello	Conosce
File system logico	Livello + vicino ai programmi.

	Nomi, directory, inode, permessi, FCB, metadati
Modulo di organizzazione	Blocchi logici, blocchi fisici, spazio libero
File system di base	Lettura e scrittura dei blocchi
Device driver	Comandi specifici dell'hardware per il controller

Il modulo di organizzazione dei file conosce la relazione tra i blocchi logici del file e i blocchi fisici del disco. Traduce i blocchi logici in blocchi fisici, decide dove allocare i nuovi blocchi e gestisce lo spazio libero, liberando i blocchi quando un file viene cancellato.

Quando parliamo di “blocco logico numero i” intendiamo la posizione del blocco all'interno del file, non la sua posizione fisica sul disco.

MOUNT TABLE = descrive i file system attualmente montati e quindi accessibili. Per ogni mount ha: dispositivo / volume, tipo di file system, punto di mount, opzioni di montaggio.

VOLUME = spazio logico di memorizzazione (partizione / insieme di dischi gestiti come una sola unità).

SYSTEM-WIDE OPEN-FILE TABLE

Si tratta della tabella globale del kernel che rappresenta i file attualmente aperti nel sistema. E' gestita dal kernel, contiene le descrizioni delle aperture effettive. Da qui ricaviamo l'FCB e raggiungiamo i blocchi dati. La system-wide open-file table contiene le informazioni relative alle aperture attive e permette a più descriptor, anche appartenenti a processi diversi, di riferirsi alla stessa apertura.

ENTRY:

- FCB / inode del file
- Modalità di apertura
- Posizione corrente (file offset)
- Contatore di riferimenti
- Altre info...

Avendo una copia dell'FCB consente di raggiungere rapidamente i metadati. Più FD possono puntare alla stessa entry, dopo dup() oppure fork() ad esempio.

PER-PROCESS OPEN-FILE TABLE

Ogni processo ha una propria tabella dei file aperti. L'indice della tabella è il file descriptor:

File descriptor	Puntatore all'apertura
0	stdin
1	stdout
2	stderr
3	entry globale del file aperto

$fd \rightarrow$ apertura del file

Le entry puntano alle corrispondenti aperture nella system-wide open-file table:

fd → tabella del processo → tabella globale → FCB → blocchi dati

FILE DESCRIPTOR = piccolo intero che identifica un file aperto all'interno di un processo.

ATTENZIONE: Il valore 3 del processo A non deve necessariamente indicare lo stesso file del valore 3 del processo B.

OPEN()

```
fd = open("/home/file.txt", O_RDONLY);
```

Il processo conosce solo il nome del file, non il suo FCB, né i blocchi fisici, quindi il kernel deve risolvere il pathname e trovare l'identificatore del file, in Unix sarebbe il numero di inode.

Per prima cosa si parte dal nome del file, quindi dobbiamo consultare la directory e passare il pathname al kernel. Il kernel percorre tutta la struttura delle directory, trova l'associazione tra file.txt e il suo inode/FCB. Carica o recupera in memoria l'FCB, crea una entry nella system-wide open file table e inserisce nella tabella del processo un puntatore a quella entry. Infine restituisce l'fd, l'indice della entry nella sua per process-open file table.

Le directory sono conservate sul disco, ma le parti usate possono essere caricate e mantenute temporaneamente in RAM.

fd = indice nella per-process open-file table

Ritorna fd quindi!

READ()

```
read(fd, buffer, n);
```

Il file è già stato aperto e identificato dal fd, quindi non dobbiamo ripetere la ricerca nella directory.

Selezioniamo tramite fd una entry nella per-process open-file table, questa punta alla system-wide open-file table, da lì recuperiamo le info per FCB e l'offset corrente. Tramite FCB individuiamo blocchi dati che vengono letti dal disco e memorizzati in un buffer del kernel, i byte richiesti vengono poi copiati nel buffer del processo (DESTINAZIONE FINALE IN USER SPACE), viene aggiornato l'offset.

Il kernel usa `fd` come indice:

fd = 3 → entry 3 della per-process open-file table

Questa tabella appartiene al singolo processo. Perciò il descriptor `3` può indicare file diversi in processi differenti.

L'entry per-processo punta a una entry della **system-wide open-file table**.

Dalla system-wide entry raggiungiamo l'FCB che ci indica quali blocchi dati contengono il file. La posizione corrente stabilisce da quale byte iniziare. Per esempio, se l'offset è 200 e vengono richiesti 100 byte, il kernel legge i byte da 200 a 299.

ESEMPIO:

Supponiamo che due processi aprano indipendentemente file.txt:

P1: `fd = open("file.txt", O_RDONLY);`

P2: `fd = open("file.txt", O_RDONLY);`

Entrambi raggiungono lo stesso FCB, dato che il file è lo stesso, ma hanno due fd DIVERSI che indicano APERTURE SEPARATE:

$$P_1 : fd\ 3 \rightarrow \text{apertura A, offset 0}$$
$$P_2 : fd\ 4 \rightarrow \text{apertura B, offset 0}$$

apertura A, apertura B $\rightarrow$ stesso FCB

Se P_1 legge 100 byte:

$$offset_A = 100, \quad offset_B = 0$$

Se invece un descriptor viene ereditato tramite `fork()` o duplicato con `dup()`, può accadere:

$$P_1 : fd\ 3 \quad e \quad P_2 : fd\ 3 \rightarrow \text{stessa apertura}$$

In questo caso condividono anche l'offset.

IMPLEMENTAZIONE DELLE DIRECTORY

La directory normalmente non contiene i dati del file, ma una serie di entry del tipo (nome, identificatore del file) che permette di raggiungere l'FCB e quindi tramite quello i blocchi dati.

2 modalità:

- **LISTA LINEARE:** entry memorizzate una dopo l'altra. Per trovare una determinata entry dobbiamo scorrere tutte le precedenti. Nel caso peggiore dobbiamo controllare tutte le

entry: $T_{ricerca} = O(n)$. Ricerca avviene sequenzialmente.

VANTAGGI: semplice da implementare

SVANTAGGI: ricerca lenta in directory grandi, inserimenti e cancellazioni diventano complessi.

- **HASH TABLE:** Si applica la funzione hash al nome.

$h(\text{nome}) = \text{posizione nella tabella}$

Per cercare `mail`, il sistema calcola direttamente:

$$h(\text{mail}) = k$$

e controlla la posizione k , evitando di scorrere tutta la directory.

Il tempo medio di ricerca è:

$T_{\text{medio}} = O(1)$

VANTAGGI: + efficiente rispetto alla lista concatenata / lista lineare, soprattutto per directory grandi

SVANTAGGI: problema delle collisioni → nomi differenti producono lo stesso valore, quindi le collisioni vanno gestite. Ad es si risolvono con il chaining: ogni posizione della tabella punta ad una lista contenente tutte le entry che hanno prodotto lo stesso hash. Richiede anche il dimensionamento della tabella e la gestione di nomi/entry di dimensione variabile.

ALLOCAZIONE DI FILE

Il file è una sequenza di blocchi logici, invece il disco è una sequenza di blocchi fisici, che devono essere allocati. Ci sono varie tecniche di allocazione:

- **ALLOCAZIONE CONTIGUA:** tutti i blocchi di un file vengono memorizzati consecutivamente sul disco.

- `start` : blocco fisico iniziale;
- `length` : numero di blocchi occupati.

$$\text{file} = (\text{start}, \text{length})$$

La sua ultima posizione è:

$$\text{ultimo blocco} = \text{start} + \text{length} - 1$$

Dati registrati in directory /FCB.

FORMULE:

Supponiamo che ogni blocco abbia dimensione B byte, che il file inizi al blocco fisico "start" e che il programma voglia accedere all'indirizzo logico LA (LA byte dall'inizio del file).

Prima dell'accesso bisogna verificare che il blocco logico Q appartenga al file:

$$Q < \text{length}$$

Equivalentemente, l'indirizzo logico deve essere inferiore alla dimensione del file:

$$LA < \text{length} \cdot B$$

Se la dimensione effettiva del file non è un multiplo del blocco, si usa la dimensione reale in byte memorizzata nell'FCB.

$$Q = \left\lfloor \frac{LA}{B} \right\rfloor$$

■ NUMERO BLOCCO LOGICO =

■ SPOSTAMENTO ALL'INTERNO DEL BLOCCO (offset):

$$R = LA \bmod B$$

■ BLOCCO FISICO :

$$\text{blocco fisico} = \text{start} + Q$$

■ INDIRIZZO FISICO COMPLETO:

$$(\text{start} + Q, R)$$

$$PA = (start + Q) \cdot B + R$$

■ INDIRIZZO FISICO ESPRESSO IN BYTE:

Supponiamo:

$$start = 19, \quad B = 512 \text{ byte}, \quad LA = 1500$$

Calcoliamo:

$$Q = \left\lfloor \frac{1500}{512} \right\rfloor = 2$$

$$R = 1500 \bmod 512 = 476$$

Quindi:

$$\text{blocco fisico} = 19 + 2 = 21$$

Il byte cercato si trova:

nel blocco fisico 21, all'offset 476

VANTAGGI:

- Accesso DIRETTO semplice: qualunque blocco può essere trovato direttamente con $\text{blocco fisico} = start + \text{blocco logico}$
- Accesso SEQUENZIALE efficiente: i blocchi sono adiacenti. In HDD questo riduce i movimenti della testina, migliora la località e consente trasferimenti di + blocchi consecutivi.
- Per rappresentare i dati bastano (start, length).

SVANTAGGI:

- Dobbiamo sempre conoscere / stimare la dimensione del blocco da allocare (rischiamo di sprecare spazio).
- Frammentazione esterna: Può esserci spazio libero complessivamente sufficiente, ma diviso in buchi piccoli e non contigui
- Se i blocchi successivi sono occupati, diventa complicato espandere il file. Per risolvere questo potremmo usare un EXTENT-BASED SYSTEM → inizialmente viene allocato uno spazio contiguo di N CLUSTER (blocchi) che hanno la STESSA DIMENSIONE. Se lo spazio non basta viene aggiunto un altro spazio contiguo detto EXTENT e ne viene salvato il puntatore.
- Compattazione (spostare i file per unire i piccoli buchi in un'unica area libera) diventa costosa.
- **ALLOCAZIONE CONCATENATA:** ogni file è rappresentato da una lista concatenata di blocchi fisici, che possono trovarsi in qualsiasi posizione del disco.

$$\text{blocco} = \text{puntatore al successivo} + \text{dati}$$

La directory conserva il puntatore al primo blocco (START) e il puntatore all'ultimo blocco (END) per rendere + veloce l'aggiunta di nuovi dati. Quando il file cresce il sistema cerca un qualunque blocco libero, inserisce il suo numero nel puntatore dell'ultimo blocco e aggiorna il campo end, non è necessario che sia adiacente ai precedenti.

VANTAGGI:

- No frammentazione esterna: i blocchi liberi possono essere ovunque, non è richiesta una regione contigua. Può esserci però frammentazione INTERNA (soprattutto nell'ultimo blocco).
- File può espandersi facilmente, basta aggiungere un nuovo blocco

SVANTAGGI:

- Accesso DIRETTO lento: per raggiungere un blocco logico Q dobbiamo partire dal primo blocco e seguire i Q puntatori. $O(n)$ = tempo di ricerca.
- Ogni blocco deve memorizzare il puntatore (riduce spazio disponibile per i dati).
- Affidabilità : se un puntatore viene danneggiato perdiamo il collegamento con tutti i blocchi successivi.

Supponiamo:

- blocco fisico di 512 byte;
- puntatore di 1 byte;
- spazio effettivo per i dati:

$$512 - 1 = 511 \text{ byte}$$

Dato un indirizzo logico LA :

$$Q = \left\lfloor \frac{LA}{511} \right\rfloor$$

$$R = LA \bmod 511$$

dove:

- Q è il numero del blocco logico da raggiungere nella catena;
- R è lo spostamento nei dati di quel blocco.

Bisogna quindi seguire la catena fino al blocco logico Q .

Nella convenzione della slide il puntatore occupa il primo byte, quindi la posizione fisica interna è:

$$R + 1$$

Il $+1$ serve a saltare il byte riservato al puntatore. In generale, con puntatore di p byte:

$$\text{dati per blocco} = B - p$$

$$Q = \left\lfloor \frac{LA}{B - p} \right\rfloor, \quad R = LA \bmod (B - p)$$

$$\text{posizione interna fisica} = R + p$$

- **FAT = File Allocation Table:** variante della lista concatenata. A differenza di questa, in cui il puntatore è memorizzato all'interno dei blocchi dati, qui sono raccolti in una tabella separata, generalmente collocata all'inizio del volume:

$$FAT[i] = \text{numero del blocco successivo a } i$$

La directory conserva il primo blocco del file.

<ESEMPIO:

test : *start* = 217

$$FAT[217] = 618$$

$$FAT[618] = 339$$

$$FAT[339] = EOF \quad \text{il file occupa: } 217 \rightarrow 618 \rightarrow 339 \rightarrow EOF$$

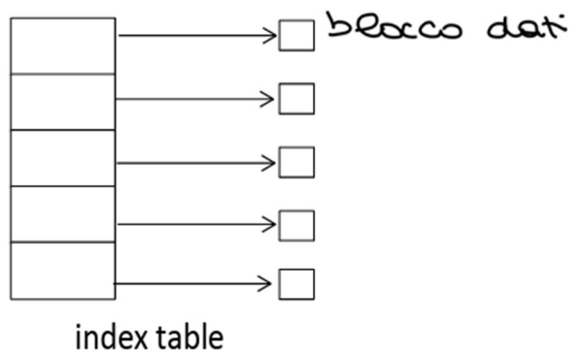
VANTAGGI:

- Può essere caricata o memorizzata in cache nella RAM, quindi non è necessario leggere ogni blocco dati dal disco, ma bisogna consultare la tabella in RAM. Il blocco fisico a questo punto può contenere più dati perché i puntatori stanno nella FAT. Più veloce di lista concatenata.

FAT in RAM $\rightarrow$ ricostruzione rapida della catena

SVANTAGGI: può diventare molto grande e compromettere molti file.

- **ALLOCAZIONE INDICIZZATA:**



Ogni file ha uno o + **BLOCCHI INDICE** (tabella di puntatori ai blocchi dati del file).

La directory memorizza il blocco fisico che contiene l'indice, non il primo blocco dati.

VANTAGGI:

- Supporta bene sia ACCESSO SEQUENZIALE che CASUALE: basta accedere al puntatore desiderato. Indice[2] → B2

SVANTAGGI:

- I blocchi indice occupano molto spazio anche quando il file è piccolo, overhead inutile. Inoltre poiché il blocco indice contiene un numero limitato di puntatore, servono + livelli.

CALCOLO DIMENSIONE MAX CON UN SOLO BLOCCO INDICE:

Se:

- blocco indice di B byte;
- puntatore di P byte;

il numero di puntatori contenuti è:

$$K = \frac{B}{P}$$

Ogni puntatore identifica un blocco dati di B byte. Quindi:

$$\text{dimensione massima} = K \cdot B$$

Nell'esempio della slide:

$$B = 512 \text{ byte}, \quad P = 1 \text{ byte}$$

$$K = \frac{512}{1} = 512 \text{ puntatori}$$

$$\text{dimensione massima} = 512 \cdot 512 = 262\,144 \text{ byte} = 256 \text{ KiB}$$

Perciò basta un solo blocco indice.

ESEMPIO TRADUZIONE:

dimensione blocco dati = $B = 512$ B

indirizzo logico = LA

Entry blocco indice = $Q = \text{arrotondamento_difetto}(LA/512)$

$Q \rightarrow \text{indice}[Q] \rightarrow \text{blocco fisico}$

$R = \text{offset nel blocco dati} = LA \bmod 512$ (resto divisione)

Indice concatenato

Supponiamo di avere blocchi da 512 word. Abbiamo 2 blocchi indice concatenati: nel primo blocco indice, l'ultima word punta al blocco indice successivo. Quindi rimangono 511 puntatori a blocchi dati.

Alla fine riusciamo a rappresentare: $511 \cdot 512$ word di dati.

$$LA = Q_1(512 \cdot 511) + R_1$$

$$R_1 = LA \bmod (512 \cdot 511) \quad Q_1 = \left\lfloor \frac{LA}{512 \cdot 511} \right\rfloor$$

Q_1 = blocco indice della catena

R_1 = posizione logica all'interno del blocco dati descritto da quel blocco indice

$$Q_2 = \left\lfloor \frac{R_1}{512} \right\rfloor$$

$$R_2 = R_1 \bmod 512$$

- Q_2 : quale entry del blocco indice selezionare;
- R_2 : offset nel blocco dati.

Il limite è che per raggiungere un blocco indice lontano bisogna attraversare la catena precedente.

INDICE A 2 LIVELLI

Abbiamo un indice ESTERNO che punta a blocchi indice INTERNI.

outer index $\rightarrow$ index block $\rightarrow$ data block

Supponiamo che ogni blocco contenga K puntatore e che abbia dimensione B .

Ogni blocco indice interno indirizza: $K \cdot B$ byte di dati.

$$Q_1 = \left\lfloor \frac{LA}{K \cdot B} \right\rfloor$$

$$R_1 = LA \bmod (K \cdot B)$$

$$Q_2 = \left\lfloor \frac{R_1}{B} \right\rfloor$$

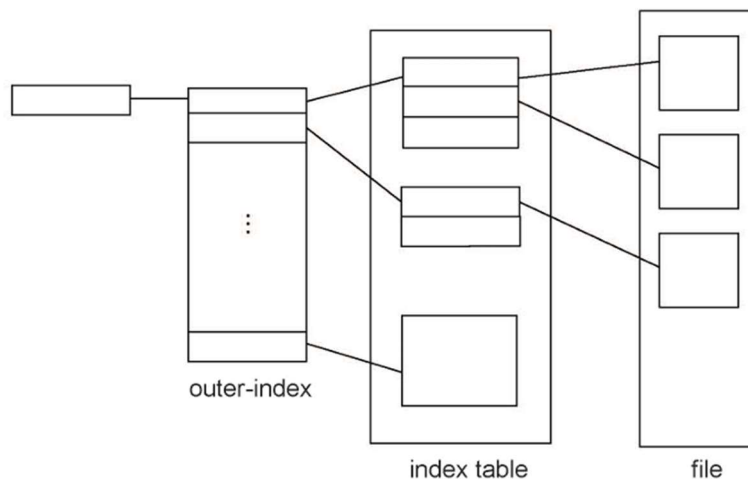
$$R_2 = R_1 \bmod B$$

Q_1 = entry del blocco indice esterno

R_1 = posizione nell'indice interno selezionato

Q_2 = entry nel blocco indice interno

R_2 = offset nel blocco dati



UNIX

Usa come FCB l'INODE, uno per ogni file. Combina allocazione diretta con allocazione indicizzata gerarchica. Ogni inode contiene:

- Metadati
- Puntatori DIRETTI
- Un puntatore INDIRETTO SINGOLO
- Un puntatore INDIRETTO DOPPIO
- Un puntatore INDIRETTO TRIPLO

PUNTATORI DIRETTI = identificano direttamente un blocco dati:

$\text{inode} \rightarrow \text{blocco dati}$

Molto efficiente per file piccoli

INDIRETTO SINGOLO: $\text{inode} \rightarrow \text{indice} \rightarrow \text{dati}$ Identifica un blocco INDICE le cui entry puntano ai blocchi dati.

INDIRETTO DOPPIO: $\text{inode} \rightarrow I_1 \rightarrow I_2 \rightarrow \text{dati}$ Il puntatore dell'inode porta a un indice esterno che punta a blocchi indice interni

INDIRETTO TRIPLO: attraversa 3 livelli di blocchi indice. Quindi possiamo indirizzare file + grandi. Ma richiedono + accessi al disco se i blocchi indice non sono già in cache.

$\text{inode} \rightarrow I_1 \rightarrow I_2 \rightarrow I_3 \rightarrow \text{dati}$

ESERCIZIO MODELLO:

Con:

$$B = 4096 \text{ byte per blocco}, \quad P = 4 \text{ byte per puntatore}$$

ogni blocco indice contiene:

$$K = \frac{B}{P} = \frac{4096}{4} = 1024$$

puntatori.

Di conseguenza:

$$\text{indiretto singolo} = K = 1024 \text{ blocchi dati}$$

$$\text{indiretto doppio} = K^2 = 1024^2 \text{ blocchi dati}$$

$$\text{indiretto triplo} = K^3 = 1024^3 \text{ blocchi dati}$$

CAPACITA' TEORICA COPERTA:

Tipo di puntatore	Blocchi dati indirizzabili	Capacità
Diretto	(1) per puntatore	(B)
Indiretto singolo	(K)	$K \cdot B = 4 \text{ MiB}$
Indiretto doppio	(K^2)	$K^2 \cdot B = 4 \text{ GiB}$
Indiretto triplo	(K^3)	$K^3 \cdot B = 4 \text{ TiB}$

Se l'inode contiene d puntatori diretti, la capacità teorica complessiva è:

$$C_{\max} = dB + KB + K^2B + K^3B$$

Tuttavia, con un file offset a 32 bit, il file può essere limitato a circa 2^{32} byte, cioè 4 GiB: in quel caso l'indiretto doppio è già sufficiente e il triplo non è completamente sfruttabile. Questo è il senso della frase della slide: la struttura potrebbe indirizzare più blocchi di quanti ne possa esprimere il puntatore al file a 32 bit.

Un file da 4 GiB con blocchi da 4 KiB contiene:

$$\frac{4 \text{ GiB}}{4 \text{ KiB}} = \frac{2^{32}}{2^{12}} = 2^{20} = 1\,048\,576$$

blocchi dati.

Ogni blocco indice interno indirizza 1024 blocchi dati, quindi servono:

$$\frac{1\,048\,576}{1024} = 1024$$

blocchi indice interni, più un blocco indice esterno.

Quindi il doppio indiretto richiede:

$$1024 \text{ indici interni} + 1 \text{ indice esterno}$$

Non serve il triplo indiretto: il doppio copre esattamente 4 GiB.

PRESTAZIONI METODI DI ALLOCAZIONE

Dipende dal tipo di accesso

- **ALLOCAZIONE CONTIGUA:** È efficiente sia in accesso sequenziale sia casuale: I blocchi sono vicini, quindi sugli HDD si riducono seek e latenza. Il problema principale è la frammentazione esterna e la crescita difficile del file.
- **ALLOCAZIONE CONCATENATA:** È adatta all'accesso sequenziale, perché si seguono i blocchi nell'ordine della catena. È poco efficiente per l'accesso casuale: per raggiungere il blocco bisogna attraversare i puntatori precedenti.
- **ALLOCAZIONE INDICIZZATA:** Supporta l'accesso diretto, ma introduce l'overhead dei blocchi indice. Se gli indici non sono in memoria, per un blocco raggiunto tramite indiretto singolo possono servire:
 - lettura del blocco indice;
 - lettura del blocco dati.

Con più livelli possono servire più letture. Cache e clustering riducono tale costo.

CLUSTERING = raggruppa blocchi vicini e li trasferisce insieme. Migliora il throughput e riduce:

- numero di richieste di I/O;
- seek sugli HDD;
- lavoro della CPU per ogni singola operazione.

Può però aumentare la frammentazione interna se parte del cluster resta inutilizzata.

Per le NVM non esistono seek o latenza rotazionale, quindi le ottimizzazioni basate sul movimento della testina sono meno importanti. Restano rilevanti il numero di operazioni, il parallelismo, la cache e il costo delle scritture.

GESTIONE DELLO SPAZIO LIBERO

Il file system deve sapere quali blocchi sono liberi e quali occupati. Per questo usa una **free-space list** che può essere realizzata in modi diversi:

- **BITMAP / BIT VECTOR:** utilizza un bit per ogni blocco / cluster del disco.
Solitamente:

$$bit[i] = \begin{cases} 1 & \text{blocco } i \text{ libero} \\ 0 & \text{blocco } i \text{ occupato} \end{cases} \quad \text{ma non è detto.}$$

Con N blocchi:

$$\text{dimensione bitmap} = \frac{N}{8} \text{ byte}$$

e:

$$N = \frac{\text{capacità del disco}}{\text{dimensione del blocco}}$$

Se l'unità di allocazione è un cluster da 4 blocchi allora il numero di cluster è

$$\frac{2^{28}}{4} = 2^{26}$$

La bitmap richiede:

$$2^{26} \text{ bit} = 2^{23} \text{ byte} = 8 \text{ MiB}$$

La bitmap facilita la ricerca di sequenze contigue di bit liberi, quindi è adatta quando servono blocchi contigui.

- **LISTA CONCATENATA DI BLOCCHI LIBERI:**

Il file system memorizza un puntatore al primo blocco libero e ogni blocco libero indica il successivo: B1 → B2 → B3...

VANTAGGI:

- Non richiede una bitmap separata
- Non spreca blocchi dati aggiuntivi perché i puntatori sono memorizzati nei blocchi liberi
- Per ottenere un singolo blocco libero, basta estrarre quello in testa ($O(1)$), quindi non serve scorrere tutta la lista

SVANTAGGI:

- Non va bene per ricercare blocchi CONTIGUI liberi perché sono memorizzati in modo sparso sul disco

Per migliorarla si usa **GROUPING** = un blocco libero contiene:

- Gli indirizzi di $n-1$ blocchi liberi
- Un puntatore ad un altro blocco contenente un nuovo gruppo di indirizzi.

Quindi con una sola lettura otteniamo + blocchi liberi:

blocco gruppo $\rightarrow [B_1, B_2, \dots, B_{n-1}, \text{prossimo gruppo}]$

quindi riduce il numero di accessi necessari.

Con il COUNTING invece sfrutta il fatto che spesso lo spazio libero forma sequenze contigue. Ogni entry contiene: (primo_blocco_libero, numero_blocchi_consecutivi).

E' una rappresentazione compatta ed è utile con allocazione contigua, extent e clustering.

Ad es con gli hard disk ha senso cercare i blocchi vicini, quindi uso GROUPING + COUNTING.

La disposizione dei blocchi influenza accesso sequenziale, casuale e frammentazione. Anche la struttura delle directory influisce: una hash table o un B+ tree è generalmente più efficiente di una ricerca lineare su directory grandi. Più informazioni vengono inserite direttamente nelle entry di directory, più alcune operazioni possono essere rapide; tuttavia le directory diventano più grandi e possono contenere dati duplicati rispetto all'FCB.

- **Preallocazione:** riserva in anticipo metadati o spazio; riduce riallocazioni, ma può sprecare memoria.
- **On demand:** assegna spazio solo quando serve; è più parsimoniosa, ma introduce più operazioni durante la crescita.

BUFFER CACHE = zona di memoria principale che conserva blocchi di disco usati + recentemente o + frequentemente. Conserva i dati sotto forma di BLOCCHI DEL DISPOSITIVO.

Quando viene richiesto un blocco:

- se è già in cache, viene restituito senza accedere al disco;
- altrimenti viene letto dal disco e inserito nella cache.

$$\text{richiesta} \rightarrow \begin{cases} \text{cache hit} & \text{nessun I/O su disco} \\ \text{cache miss} & \text{lettura dal disco} \end{cases}$$

E' diversa dalla PAGE CACHE che invece conserva i dati dei file sotto forma di PAGINE DI MEMORIA VIRTUALE. E' usata soprattutto con i file mappati in memoria con mmap(). Una parte del file viene associata allo spazio di indirizzamento virtuale del processo. Quando il processo accede ad una pag che non sta in RAM si verifica comunque un page fault e il SO carica dal file la pag corrispondente.

Queste 2 portano ad un problema, detto delle CACHE SEPARATE. Poiché non esiste una cache unificata esistono 2 percorsi:

- Mmap() $\rightarrow$ page cache()
- Read()/write() $\rightarrow$ buffer cache

Quindi lo stesso dato di un file potrebbe quindi essere presente contemporaneamente come pagina nella page cache o come blocco nella buffer cache (DOUBLE CACHING, che comporta consumo inutile di RAM, copie aggiuntive tra le cache, rischio incoerenza tra le 2 versioni, > complessità per sincronizzazione).

Per risolvere il problema del Double Caching usiamo una UNIFIED BUFFER CACHE che utilizza un'unica cache (organizzata in pagine normalmente) sia per MEMORY-MAPPED I/O che per I/O ordinario con read() e write().

Il percorso diventa:

$$\begin{cases} mmap() \\ read()/write() \end{cases} \rightarrow \text{cache unificata} \rightarrow \text{file system} \rightarrow \text{disco}$$

In questo modo lo stesso dato possiede una sola copia in RAM (un'unica entry in cache → una pag del file). Riduciamo il consumo di memoria, evitiamo copie tra page cache e buffer cache, manteniamo coerenti i 2 percorsi, semplifichiamo la gestione dei dati modificati. E' difficile però adottare una specifica politica di rimpiazzo.

SCRITTURA SINCRONA: la syscall viene completata solo DOPO che il dato è stato effettivamente scritto sul dispositivo stabile (disco). Più lenta ma più sicura

SCRITTURA ASINCRONA: i dati vengono copiati nella cache e la chiamata può terminare prima della scrittura fisica sul disco, sarà poi il kernel a scriverli successivamente. Rischiamo di perdere dati non ancora persistiti in caso di stop alimentazione o di crash, ma è più veloce perché possiamo raggruppare + scritture, riordinarle e scrivere insieme blocchi consecutivi.

Per ottimizzare l'accesso sequenziale abbiamo 2 tecniche:

- **READ-AHEAD:** Quando il sistema legge il blocco , prevede che serviranno anche i successivi e li carica anticipatamente: $B_i \rightarrow \text{precarica } B_{i+1}, B_{i+2}, \dots$. Se la previsione è corretta, le letture successive trovano i dati già in cache.
- **FREE-BEHIND:** Dopo aver letto e utilizzato un blocco in una scansione sequenziale, il sistema lo rimuove presto dalla cache, perché è improbabile che venga riutilizzato.

Per evitare di perdere i dati dobbiamo avere ridondanti (BACK-UP).

Prima di modificare realmente le strutture del file system, il sistema registra nel **log** tutte le modifiche da eseguire. Il log è una zona riservata del disco, chiamata anche **journal**, nella quale le informazioni vengono scritte sequenzialmente.

Un'operazione, come la creazione di un file, richiede più modifiche:

$$T = \{\text{directory, inode, bitmap}\}$$

La sequenza è:

transazione → scrittura sequenziale nel log → commit → aggiornamento del



La transazione è **committed** quando è stata scritta completamente e persistentemente nel log. Le strutture definitive del file system potrebbero però non essere ancora aggiornate.

Dopo il commit abbiamo l'aggiornamento del file system. le modifiche vengono applicate **asincronamente** a directory, inode e bitmap. Quando l'aggiornamento è completo, lo spazio occupato dalla transazione nel log può essere riutilizzato. In caso di crash:

- transazione senza commit → viene ignorata perché incompleta;
- transazione con commit ma non completamente applicata → viene rieseguita tramite **journal replay**.

Il vantaggio è un recovery rapido: il sistema controlla solamente le transazioni rimaste nel log, evitando una scansione completa del file system e impedendo aggiornamenti parziali dei metadati.

Struttura / caso	Blocchi contigui necessari?	Complessità	Valutazione
Free list – 1 blocco	No	$O(1)$	Si preleva il blocco indicato da head_ptr e si aggiorna la testa
Free list – (N) blocchi qualsiasi	No	$O(N)$	Si estraggono i primi (N) blocchi della lista
Free list non ordinata – (N) blocchi contigui	Sì	$O(N \cdot N_F)$ nel caso peggiore	Bisogna cercare tra indirizzi disordinati una sequenza di (N) blocchi consecutivi
Free list ordinata – (N) blocchi contigui	Sì	$O(N_F)$	Scorrendo gli indirizzi ordinati si riconoscono facilmente le sequenze consecutive
Bitmap semplice – 1 blocco	No	$O(N_F)$ nel caso peggiore	Bisogna cercare un bit libero; in pratica le operazioni per word rendono la ricerca veloce
Bitmap semplice – (N) blocchi contigui	Sì	$O(N_F)$	Si cerca una sequenza di (N) bit liberi consecutivi
Bitmap gerarchica – (N) blocchi contigui	Sì	Circa $O(\log N_F)$	Il costo logaritmico richiede strutture ausiliarie; non vale per una bitmap semplice
Memoria paginata	No	$O(1)$ per frame con free list	I frame assegnati alle pagine possono essere fisicamente non contigui
Memoria del kernel	Spesso sì	Dipende dalla struttura	Per regioni fisicamente contigue sono preferibili bitmap, buddy system o strutture ordinate
File gestito tramite inode	No	$O(1)$ per nuovo blocco con free list	L'inode può puntare a blocchi sparsi; non richiede allocazione contigua
File gestito tramite FAT	No	$O(1)$ con head_ptr o free list separata	I blocchi possono essere sparsi e vengono collegati tramite la FAT
Allocazione contigua / extent	Sì	Bitmap: $O(N_F)$; free list ordinata: $O(N_F)$	Serve trovare una sequenza di blocchi fisicamente adiacenti
Scelta generale	—	—	Free list per ottenere rapidamente blocchi qualsiasi;

			bitmap o strutture ordinate per cercare blocchi contigui
--	--	--	--

ottenere blocchi liberi **qualsiasi** da una free list costa , non . Il costo elevato nasce quando i blocchi devono essere anche **contigui**.

CH 10

Durante l'esecuzione, un processo non utilizza contemporaneamente tutto il proprio codice e tutti i propri dati. Alcune parti, come routine di errore, funzioni rare o grandi strutture dati, potrebbero non essere mai richiamate. La memoria virtuale permette quindi l'**esecuzione parzialmente caricata**: nella RAM si mantengono soltanto le parti attualmente necessarie (logica simile al dynamic loading ma la MEMORIA VIRTUALE è direttamente gestita da SO + HW).

VANTAGGI:

- il programma non è limitato dalla dimensione della RAM
- possono risiedere più processi contemporaneamente in memoria
- riduzione tempo di avvio
- aumenta grado di multiprogrammazione (+ programmi runnano insieme perché ogni programma occupa meno memoria)
- aumenta utilizzo della CPU + throughput
- diminuisce l'I/O necessario per caricare i programmi
- (tempo di risposta NON aumenta necessariamente)
- Ogni processo occupa – frame

La MEMORIA VIRTUALE (intervallo di indirizzi logici) separa lo spazio degli INDIRIZZI LOGICI /VIRTUALI generati dalla CPU e visibili al processo, dallo spazio di INDIRIZZI FISICI (indirizzi reali della RAM).

Ogni processo vede uno spazio virtuale privato, generalmente continuo e con origine dall'indirizzo 0, anche se le sue pagine si trovano in frame fisici non contigui.

La traduzione è effettuata dalla MMU tramite la page table. La **page table** associa ciascuna pagina virtuale a un frame fisico: VPN (Virtual Page Number) → PFN (Physical Frame Number).

Può essere che: SPAZIO DI INDIRIZZAMENTO LOGICO > RAM.

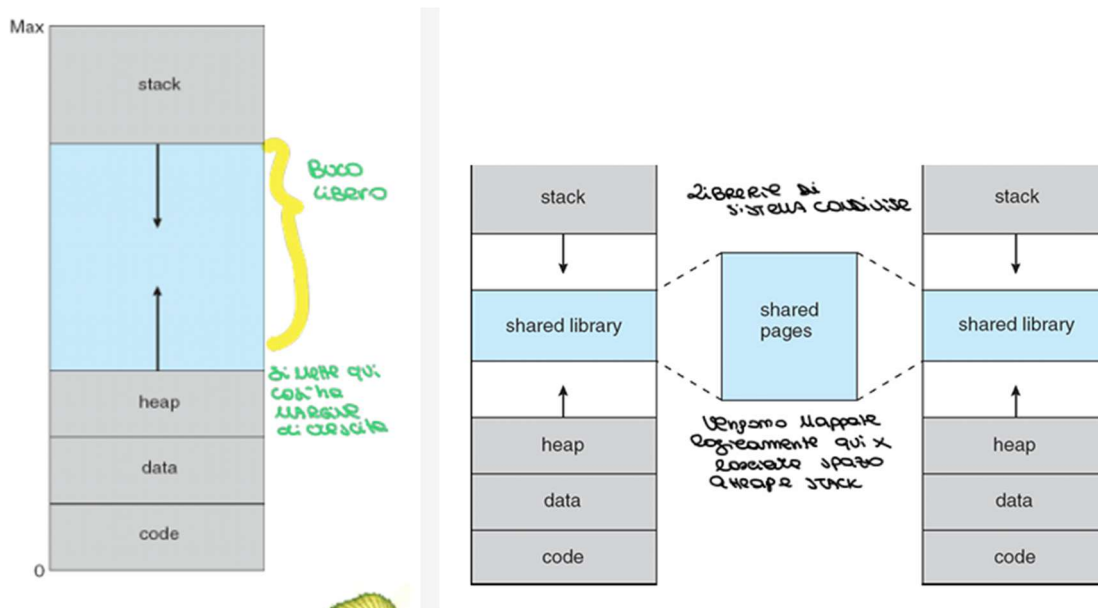
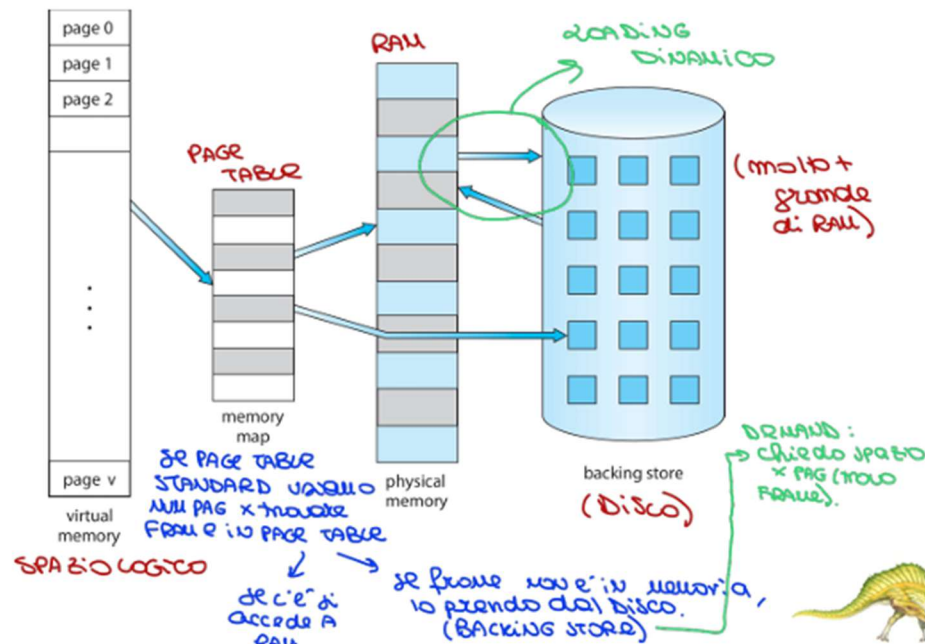
- SPAZIO VIRTUALE → diviso in PAGINE
- RAM → divisa in FRAME
- DISCO → contiene le pagine non residenti

Quindi una pagina può essere:

- **presente**: la page table contiene il frame fisico;
- **non presente**: la pagina si trova nel backing store;
- **non valida**: non appartiene allo spazio virtuale del processo.

Lo spazio virtuale è generalmente organizzato in : CODE + DATA (indirizzi + bassi), HEAP (cresce), BLOCCO LIBERO, STACK (Parte da indirizzi + alti e cresce verso indirizzi minori). Tra heap e stack c'è un BUCO VIRTUALE: gli indirizzi esistono ma non sono associati a FRAME FISICI. In questo caso abbiamo uno SPARSE ADDRESS SPACE (spazio virtuale contenente regioni non allocate fisicamente). Questi BUCHI consentono la crescita dinamica di heap +

vehere contiguo
molto + grosso dello RAM



DEMAND PAGING

= Il **demand paging** è una tecnica di memoria virtuale in cui le pagine di un processo vengono caricate in RAM **solo quando vengono effettivamente richieste**. Non si carica quindi tutto il

processo al momento dell'avvio, può partire avendo in RAM solo poche pag o persino nessuna (PURE DEMAND PAGING).

PAGER = componente che trasferisce le singole pagine DA DISCO A RAM (SWAP OUT/SWAP IN). INDOVINA (fa una previsione futura) di quali pagine verranno usate prima di fare SWAP OUT. Viene definito LAZY SWAPPER perché carica una pag solo quando diventa necessaria. Tradizionalmente lo swapper trasferisce interi processi invece.

VANTAGGI:

- meno operazioni di I/O, perché non si caricano pagine inutilizzate;
- minore occupazione della RAM;
- avvio più rapido del processo;
- più processi contemporaneamente residenti in memoria.

Quando il processo riferisce una pagina virtuale, la MMU consulta la relativa entry della **page table**. Si possono verificare tre casi:

1. **Pagina presente in RAM:** la MMU ottiene il numero del frame e l'accesso procede normalmente.
2. **Pagina appartenente al processo ma non residente:** viene generato un **page fault**.
3. **Indirizzo non appartenente allo spazio del processo:** il riferimento è illegale e il processo viene terminato.

Un **page fault** è un'eccezione generata quando la pagina richiesta appartiene al processo, ma non è presente nella RAM.

Ogni entry della page table contiene un bit di stato:

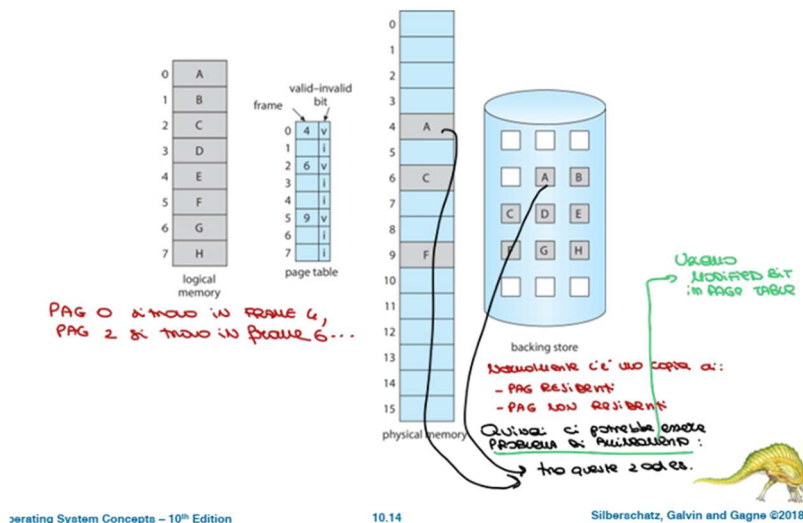
- v (**valid**): la pagina è residente in RAM e l'entry contiene un frame valido;
- i (**invalid**): la pagina non è attualmente residente oppure il riferimento non appartiene allo spazio logico del processo.

Quando la MMU trova i, genera una **trap di page fault** e trasferisce il controllo al sistema operativo.



Page Table When Some Pages Are Not in Main Memory

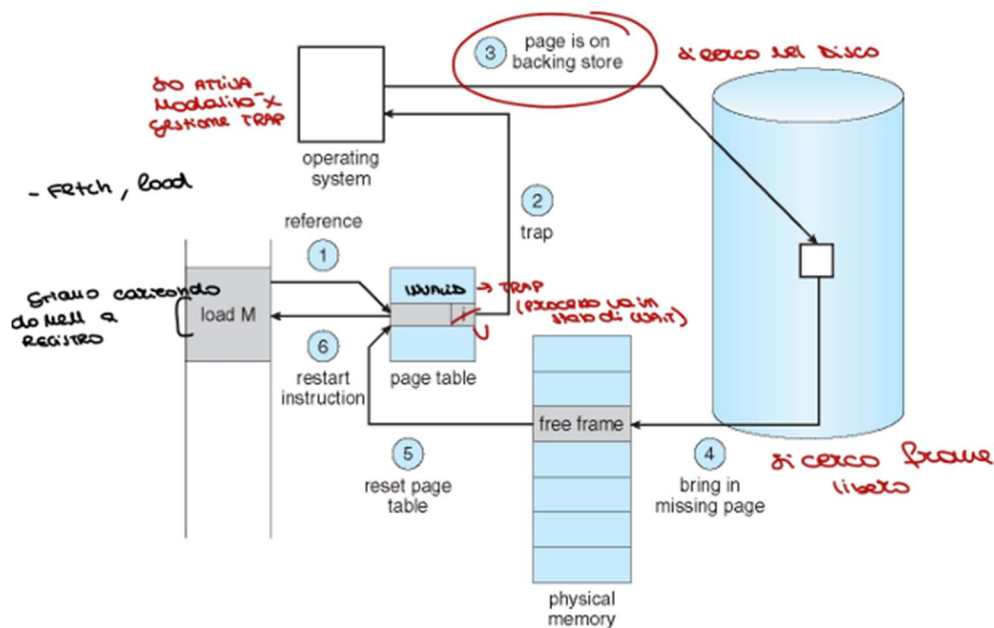
PAGE FAULT



Se il processo prova ad accedere alla pag virtuale 3 (si trova nel backing store, come pag 1,5,7) che ha bit i, da page fault.

Supponiamo che un'istruzione, come load M, acceda a una pagina non residente:

1. La MMU trova i nella page table e genera una **trap**.
2. La CPU passa dalla modalità utente alla modalità kernel ed esegue il page-fault handler.
3. Il sistema operativo controlla il riferimento:
 - se è illegale, termina il processo;
 - se la pagina è valida ma non residente, continua.
4. Individua la pagina nel **backing store**.
5. Preleva un frame dalla **free-frame list**.
6. Avvia un'operazione di lettura per trasferire la pagina dal disco al frame.
7. Aggiorna la page table inserendo il numero del frame e impostando il bit a v.
8. Ripristina lo stato del processo.
9. Riavvia dall'inizio l'istruzione che aveva causato il page fault.



CASO ESTREMO → PURE DEMAND PAGING: avviare un processo con 0 PAG CARICATE IN RAM. Il SO imposta il PC sull'indirizzo virtuale della prima istruzione, ma la pag che la contiene non è residente. Quando la CPU tenta il fetch dell'istruzione: la MMU trova il bit INVALID, genera PAGE FAULT, il SO carica dal disco la pag contenente l'istruzione e questa viene riavviata. Stessa cosa accade al primo accesso ad una nuova pag del processo. Una sola istruzione talvolta può accedere a + pag differenti e causare + page fault.

$$C = A + B$$

L'esecuzione può richiedere:

- la pagina contenente l'istruzione, per il fetch;
- la pagina contenente A ;
- la pagina contenente B ;
- la pagina contenente C , per salvare il risultato.

Se tutte queste pagine sono non residenti, la stessa istruzione può provocare fino a quattro page fault successivi. Dopo ogni fault, l'istruzione viene riavviata dall'inizio, finché tutte le pagine necessarie sono presenti.

I PAGE FAULT sono ridotti dalla LOCALITA' DEI RIFERIMENTI: urante un intervallo di tempo, un processo tende ad accedere ripetutamente alle stesse pagine o a pagine vicine. Dopo i page fault iniziali, le pagine del codice e dei dati utilizzati più frequentemente rimangono residenti in RAM. Di conseguenza, gli accessi successivi non generano nuovi fault.

Il demand paging richiede:

- PAGE TABLE con valid/invalid bit per indicare se la pag è residente
- MEMORIA SECONDARIA con SWAP SPACE (per conservare le pag non residenti)
- SUPPORTO HW per PF
- Instruction restart.

La **free-frame list** è la struttura con cui il sistema operativo mantiene l'elenco dei **frame fisici liberi**. Quando avviene un PF il SO preleva un frame libero dalla lista, lo azzera eliminando i dati precedenti (ZERO-FILL-ON-DEMAND), vi carica la pag richiesta dal disco. All'avvio del sistema,

tutti i frame disponibili vengono inseriti nella free-frame list. Questa può essere implementata con: LISTA CONCATENATA, BITMAP.

Gestione completa di un page fault (caso peggiore, I/O)

1. La MMU rileva una pagina non residente e genera una **trap** al sistema operativo.
2. Il SO salva registri e stato del processo.
3. Verifica che si tratti di un page fault.
4. Controlla il riferimento:
 - illegale → termina il processo;
 - valido → individua la pagina sul disco.
5. Preleva un frame libero e avvia la lettura dal disco.
6. Durante l'I/O, il processo viene bloccato e la CPU assegnata a un altro processo.
7. Terminato il trasferimento, il disco genera un **interrupt**.
8. Il SO salva lo stato del processo attualmente in esecuzione e riconosce l'interrupt del disco.
9. Aggiorna la page table: inserisce il frame e imposta il bit a valid.
10. Il processo diventa **ready** e attende nuovamente la CPU.
11. Quando viene schedulato, il SO ripristina il suo stato e **riavvia l'istruzione interrotta**.

COSTO DI PF → gestione della trap + lettura pagina da disco + scrittura sul disco della pag sostituita + riavvio del processo. (EAT = Effective Access Time)

$P = 0 \rightarrow$ no pf

$P = 1 \rightarrow$ tutti i references sono pf

Indicando con p la probabilità di page fault:

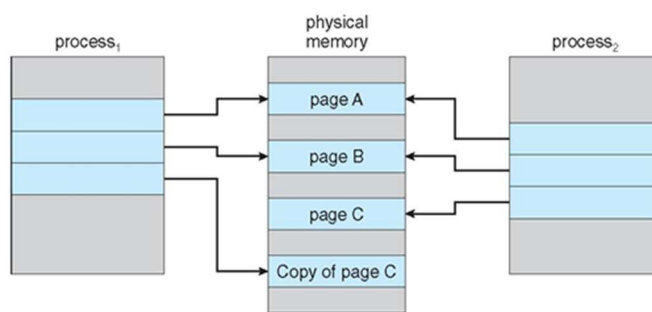
$$EAT = (1 - p) \cdot T_{mem} + p \cdot T_{page\ fault}$$

con $0 \leq p \leq 1$.

- Memory access time = 200 nanoseconds *(dovrebbe essere EAT che tiene conto di TLB)*
- Average page-fault service time = 8 milliseconds *se no PAGE FAULT = no dirty TLB miss + Invalid in PAGE TABLE.*
- $EAT = (1 - p) \times 200 + p (8 \text{ milliseconds})$
 $= (1 - p) \times 200 + p \times 8,000,000$ *trascurabile*
 $= 200 + p \times 7,999,800$
- If one access out of 1,000 causes a page fault, then $\rightarrow p = \frac{1}{1000}$
 $EAT = 8.2 \text{ microseconds.}$ *(1 page fault ogni 1000 Access.)*
This is a slowdown by a factor of 40!!
- If want performance degradation < 10 percent *10%*
 - $220 > 200 + 7,999,800 \times p$ *EAT = 220*
 $20 > 7,999,800 \times p$
 - $p < .0000025$
 - < one page fault in every 400,000 memory accesses

OTTIMIZZAZIONI

- SWAP SPACE $\rightarrow$ + veloce del file system perché non deve gestire metadati + directory. Le pagine di codice sono già nel file eseguibile quindi possono essere caricate direttamente da lì, oppure eliminate senza swap-out perché in casi estremi posso rileggerle direttamente dall'exe. Invece devo salvare nello swap le PAG ANONIME (stack + heap), o le PAG MODIFICATE. Mobile system NON SUPPORTANO SWAPPING
- COW $\rightarrow$ Dopo fork(), padre e figlio hanno spazi virtuali distinti, ma inizialmente le rispettive page table puntano agli **stessi frame fisici**, marcati read-only. Se uno dei 2 processi modifica una pag, viene generata una trap, il SO crea una copia della pag, assegna la copia al processo scrivente, l'altro mantiene la pag originale. Così evitiamo di copiare tutte le pag ma copiamo solo quelle effettivamente modificate (velocissimo tempo di avvio).



*se uno dei 2 modifica una PAG (PAG C) $\rightarrow$
 si crea copia (COW), e l'altro mantiene
 copia vecchia*

Se avviene un page fault e la free-frame list è vuota, il sistema operativo deve eseguire il **page replacement**: sceglie una pag residente come **VITTIMA**, libera il relativo frame e carica nel frame la pag richiesta (impedisce la saturazione della memoria fisica). L'algoritmo deve scegliere una pagina poco utilizzata, per ridurre la probabilità che venga richiesta nuovamente subito dopo.
 Ogni pagina possiede un **dirty bit**:

- dirty = 0: pagina non modificata; può essere eliminata senza scriverla sul disco;
- dirty = 1: pagina modificata; deve essere salvata con uno swap-out prima della sostituzione.

Il dirty bit riduce quindi il numero di trasferimenti verso il disco.

PROCEDURA

1. Individua sul disco la pagina richiesta.
2. Cerca un frame libero.
3. Se non esiste, sceglie una pagina vittima.
4. Imposta come invalid la sua entry.
5. Se la vittima è modificata, la salva sul disco.
6. Carica la nuova pagina nel frame liberato.
7. Aggiorna page table e frame table.
8. Riavvia l'istruzione che aveva causato il fault.

Nel caso peggiore servono due trasferimenti:

swap-out della vittima + swap-in della pagina richiesta

R	M	Significato	Preferenza come vittima
0	0	Non usata, non modificata	Migliore
0	1	Non usata, modificata	Seconda
1	0	Usata, non modificata	Terza
1	1	Usata e modificata	Peggiora

Sono necessari due algoritmi distinti per gestire i PF:

- **Frame-allocation algorithm:** stabilisce quanti frame assegnare a ogni processo.
- **Page-replacement algorithm:** stabilisce quale pagina sostituire quando non ci sono frame liberi.

L'obiettivo è minimizzare i page fault sia al primo accesso sia nei successivi riaccessi. Gli algoritmi vengono valutati usando una **reference string**, cioè la sequenza dei numeri di pagina richiesti a cui si accede: 7,0,1,2,5,3,2,1,1,1,8...

Se una pagina è già residente, non avviene alcun fault. Altrimenti si verifica un page fault. Il risultato dipende anche dal numero di frame disponibili m: normalmente, **aumentando m**, **diminuiscono i fault**.

La frequenza media dei page fault di un algoritmo A è:

$$f(A, m) = \sum_w p(w) \frac{F(A, m, w)}{\text{len}(w)}$$

dove F è il numero di fault prodotti sulla reference string w . L'algoritmo ottimo sceglierebbe la pagina usata più lontano nel futuro, ma non è realizzabile perché non conosciamo gli accessi futuri.

w = reference string

ALGORITMO FIFO

FIFO sostituisce la pagina che si trova in memoria da più tempo, cioè quella caricata per prima. Attenzione: non considera quando una pagina è stata usata l'ultima volta. **Considera soltanto quando è entrata in RAM.**

Esempio iniziale

Reference string:

7, 0, 1, 2, 0, 3, ...

Tre frame inizialmente vuoti:

Riferimento	Frame dopo l'accesso	Risultato	Vittima
7	7 --	Fault	—
0	7 0 —	Fault	—
1	7 0 1	Fault	—
2	2 0 1	Fault	7
0	2 0 1	Hit	—
3	2 3 1	Fault	0
0	2 3 0	Fault	1

Si implementa con una coda circolare con HEAD che indica la prossima pag da sostituire. Quando arriva una nuova pagina, viene inserita nella posizione indicata da HEAD e questo viene fatto avanzare al frame successivo.

Il problema di FIFO è che non considera l'utilità futura o recente della vittima, ma solo il suo iniziale caricamento. Elimina la pagina caricata da più tempo, anche se è stata appena usata. Per questo si verifica l'ANOMALIA DI BELADY:

Normalmente ci aspetteremmo:

più frame $\Rightarrow$ meno page fault

Con FIFO non è sempre vero: aumentando i frame, i page fault possono aumentare.

Questo fenomeno è chiamato **anomalia di Belady**.

ALGORITMO OPT (OTTIMO)

OPT sostituisce la pagina il cui prossimo utilizzo si trova più lontano nel futuro.

Quando si verifica un fault si osservano le pag presenti nei frame, si cerca nella parte futura della reference string quando sarà nuovamente usata quella pag, si elimina quella usata + tardi. Se una pag non verrà mai più usata, è la vittima migliore. Tengo le pagine che mi serviranno prima ed elimino quella che mi servirà più tardi.

Supponiamo che i frame contengano:

[7, 0, 1]

e che debba entrare 2. La parte futura della stringa è:

0, 3, 0, 4, 2, 3, ..., 7, ...

Prossimi utilizzi:

- 0 sarà usata subito;
- 1 sarà usata più avanti;
- 7 sarà usata molto più tardi.

OPT elimina quindi 7.

PROBLEMA: non è implementabile. Il sistema operativo non conosce in anticipo i riferimenti futuri di un processo. OPT viene quindi utilizzato come **termine di confronto**: se un algoritmo si avvicina a OPT, è efficiente; nessun algoritmo può produrre meno fault di OPT sulla stessa stringa e con lo stesso numero di frame.

LRU (Least Recently Used)

LRU cerca di approssimare OPT utilizzando il passato. **Sostituisce la pagina che non viene utilizzata da più tempo, cioè quella con l'accesso più lontano nel passato. Una pagina usata recentemente ha buone probabilità di essere usata nuovamente presto.**

Esempio iniziale

Reference string:

7, 0, 1, 2, 0, 3, ...

Riferimento	Frame	Risultato	Vittima
7	7 --	Fault	—
0	7 0 -	Fault	—
1	7 0 1	Fault	—
2	2 0 1	Fault	7
0	2 0 1	Hit	—
3	2 0 3	Fault	1

Con LRU dobbiamo conoscere esattamente l'ordine degli ultimi accessi. Ogni entry della PT contiene un CONTATORE, un TIMESTAMP. Quando bisogna scegliere una vittima, si elimina la pagina con il timestamp più piccolo, perché è stata usata meno recentemente. Dobbiamo aggiornare questo contatore ad ogni accesso. COSE NEGATIVE: per trovare il timestamp / contatore + piccolo dobbiamo cercare tra TUTTE LE PAG RESIDENTI + serve supporto HW.

Inoltre, serve uno STACK LRU: si mantiene una LISTA DOBBIAMENTE CONCATENATA. In TESTA abbiamo la pag usata + recentemente. In FONDO abbiamo la pag usata - recentemente. A ogni riferimento, la pagina viene spostata in testa, anche se era già residente.

ESEMPIO: prima di riferire una pag lo stack è: 2,1,0,7,4. Riferiamo 7 → 7,2,1,0,4. La vittima LRU è sempre l'ultima, quindi 4. Quindi non serve cercare la vittima e questo è un VANTAGGIO (estrazione in coda). Svantaggio: ogni accesso richiede la rimozione e il reinserimento della pagina nella lista, quindi è costoso.

LRU non presenta l'anomalia di Belady perché è uno **stack algorithm**.

Con m frame, l'insieme delle pagine residenti è sempre contenuto nell'insieme ottenuto con $m + 1$ frame:

$$S_m \subseteq S_{m+1}$$

Aggiungere un frame non può quindi aumentare i page fault. Anche OPT possiede questa proprietà; FIFO no.

Approssimazione LRU con REFERENCE BIT

LRU esatto è costoso. Per questo si utilizza spesso un bit hardware chiamato **reference bit** o **accessed bit**, indicato con R . Inizialmente per ogni pagina abbiamo $R=0$, quando la pagina viene acceduta l'hw imposta $R=1$. Periodicamente il SO può riportarlo a 0.

- $R=1 \rightarrow$ pag usata recentemente
- $R=0 \rightarrow$ nessun uso rilevato nell'intervallo recente.

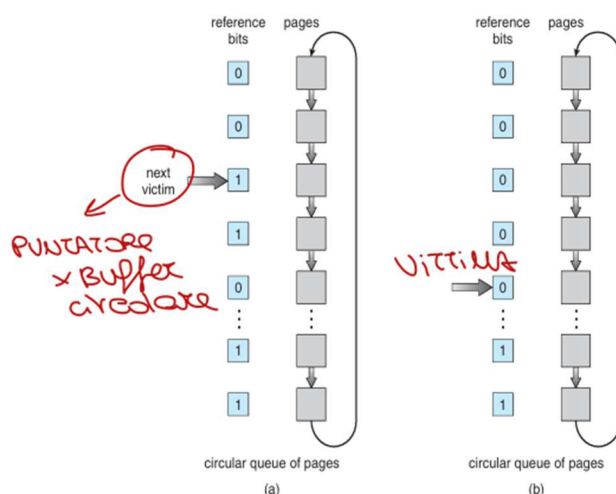
Quando serve una vittima si preferisce una pagina $R=0$.

LIMITE = il singolo bit non ci indica un ordine preciso, ci dice solo quale pag sono state usate prima o dopo.

SECOND CHANCE

Combina FIFO + REFERENCE BIT R . Le pagine sono organizzate in una coda circolare e un puntatore, chiamato **clock hand**, indica la prossima possibile vittima. Quando serve un frame controlla la pagina puntata:

- Se $R=0 \rightarrow$ la pag non è usata recentemente, quindi viene scelta come vittima, la nuova pag occupa il suo frame e il puntatore avanza
- Se $R=1 \rightarrow$ la pag è stata usata recentemente, quindi riceve una seconda possibilità: il bit $R=0$ (viene azzerato), la pag rimane in memoria e il puntatore avanza.



Second Chance: esempio sintetico

Reference string iniziale:

7, 0, 1, 2, 0, 3

Tre frame, con pagina^R .

Rif.	Frame	Caso
7	$[7^1, -, -]$	Fault: frame libero
0	$[7^1, 0^1, -]$	Fault: frame libero
1	$[7^1, 0^1, 1^1]$	Fault: frame libero
2	$[2^1, 0^0, 1^0]$	Fault: tutti hanno $R = 1$, vengono azzerati; 7 viene sostituita
0	$[2^1, 0^1, 1^0]$	Hit: 0 è presente, quindi $R(0) = 1$
3	$[2^1, 0^0, 3^1]$	Fault: 0 ha $R = 1$, riceve una seconda chance; 1 ha $R = 0$ e viene sostituita

ENHANCED SECOND CHANCE

Usa 2 bit:

- REFERENCE BIT (R) $\rightarrow$ 1 se pag usata recentemente
- MODIFY /DIRTY BIT M $\rightarrow$ 1 se pag è stata modificata e deve essere salvata prima della rimozione.

Classe	Significato	Convenienza
(0,0)	Non recente e non modificata	Vittima migliore
(0,1)	Non recente ma modificata	Va prima salvata
(1,0)	Recente e pulita	Potrebbe servire presto
(1,1)	Recente e modificata	Vittima peggiore

Il puntatore Clock scorre circolarmente e cerca una pagina nella **classe non vuota più bassa**. Si preferisce (0,0) perché non è recente e può essere eliminata senza scriverla sul disco.

Enhanced Second Chance: casi significativi

Caso	Pagine nei frame (R,M)	Vittima	Perché
1. Presente (0,0)	A(1,1) , B(0,1) , C(1,0) , D(0,0)	D	Non è recente né modificata: si elimina senza scrittura su disco.
2. Nessuna (0,0)	A(1,1) , B(0,1) , C(1,0)	B	Non è recente, ma è dirty: prima della sostituzione serve lo swap-out.
3. Tutte recenti	A(1,1) , B(1,0) , C(1,1)	Dopo l'azzeramento di R , una pagina pulita	Il Clock pone $R = 0$. Gli stati diventano A(0,1) , B(0,0) , C(0,1) ; quindi sceglie B, perché ora è (0,0) .

Ordine di preferenza

$$(0, 0) \rightarrow (0, 1) \rightarrow (1, 0) \rightarrow (1, 1)$$

ALGORITMI BASATI SUL CONTEGGIO

Il sistema mantiene, per ogni pagina, un contatore del numero di riferimenti ricevuti.

- **LFU – Least Frequently Used:** elimina la pagina con il contatore più basso, quindi quella usata meno frequentemente.
- **MFU – Most Frequently Used:** elimina quella con il contatore più alto. L'idea è che una pagina con conteggio basso potrebbe essere appena entrata e non aver ancora avuto tempo di essere utilizzata.

Problema: un conteggio elevato accumulato nel passato non garantisce che la pagina sia ancora utile. Per questo tali algoritmi sono poco utilizzati.

PAGE BUFFERING

Il **page buffering** mantiene sempre una riserva di frame liberi, così un page fault non deve aspettare la scelta e la liberazione immediata di una vittima. Così quando mi servono ce li ho.

Quando avviene un fault:

1. la pagina richiesta viene caricata subito in un frame libero;
2. separatamente viene scelta una vittima;
3. la vittima viene rimossa e il suo frame reinserito nel pool.

Ulteriori ottimizzazioni:

- le pagine dirty vengono salvate anticipatamente quando il disco è libero;
- il frame liberato può mantenere temporaneamente il vecchio contenuto;
- se la stessa pagina viene richiesta prima del riutilizzo del frame, può essere recuperata senza leggerla nuovamente dal disco.

Serve quindi a ridurre la latenza del page fault e a correggere rapidamente una sostituzione sbagliata. Gli algoritmi del sistema operativo cercano di prevedere gli accessi futuri osservando quelli passati. Alcune applicazioni, come i database, conoscono però meglio il proprio schema di accesso. Può inoltre verificarsi il **double buffering**: la stessa informazione viene conservata sia nel buffer dell'applicazione sia nella page cache del sistema operativo, occupando memoria due volte. Questo capita soprattutto per app memory-intensive.

ALLOCAZIONE FRAME

Ogni processo deve ricevere un **numero minimo di frame**, necessario per eseguire almeno un'istruzione completa. Il massimo assegnabile è invece limitato dal numero totale di frame disponibili nel sistema. Si possono usare 2 schemi: allocazione fissa, o priority allocation.

ALLOCAZIONE FISSA → il numero di frame assegnato ad ogni processo viene stabilito in anticipo. Può essere:

- **UNIFORME/EQUA**: i frame vengono divisi in parti uguali. E' semplice ma non considera le diverse dimensioni dei processi. Ad esempio se ho 100 frame e 5 processi, per ogni processo avrò $100/5 = 20$ frame per processo.
- **PROPORZIONALE**: ogni processo riceve frame proporzionalmente alla sua dimensione.

$$a_i = \frac{s_i}{S} \cdot m$$

si = dimensione processo i, S = dimensione complessiva (somma di tutti i processi), m = frame disponibili, ai = frame assegnati al processo.

Con $s_1 = 10$, $s_2 = 127$ e 62 frame utilizzabili:

$$a_1 = \frac{10}{137} \cdot 62 \approx 4$$

$$a_2 = \frac{127}{137} \cdot 62 \approx 57$$

Il processo più grande riceve quindi più frame.

Abbiamo 2 tipi di REPLACEMENT:

- **GLOBAL** → un processo può scegliere una vittima tra tutti i frame disponibili, anche quelli appartenenti ad altri processi. **VANTAGGI**: throughput >>, migliore utilizzo della memoria. **SVANTAGGI**: prestazioni del singolo processo meno prevedibili, dato che dipende dagli altri.
- **LOCALE** → Ogni processo può sostituire soltanto pagine appartenenti ai propri frame. **VANTAGGI**: il processo non sottrae frame agli altri, prestazioni + stabili. **SVANTAGGIO**: alcuni frame sono sottoutilizzati.

ESEMPIO STRATEGIA PREVENTIVA DI GLOBAL REPLACEMENT: RECLAIMING. Il sistema cerca di mantenere sempre una riserva di frame nella **free-frame list**. Le richieste di memoria vengono soddisfatte usando questa lista, senza aspettare che diventi vuota. Il sistema usa due soglie:

- **minimum threshold**: se i frame liberi scendono sotto questa soglia, il kernel avvia il reclaiming e seleziona pagine da rimuovere, libera frame
- **maximum threshold**: quando i frame liberi raggiungono questa soglia, il kernel sospende il reclaiming.

NUMA (Non-Uniform Memory access)

Si parla di tempo di accesso alla memoria non uniforme, perché dipende dalla posizione della RAM rispetto alla CPU. Nei sistemi multiprocessore (con molte CPU), la RAM è distribuita su + nodi. Quindi ogni nodo contiene una o + CPU + una parte di RAM. Tutte le CPU possono accedere a tutta la memoria ma con una velocità differente. Tipo se CPU è vicina a MEMORY, l'accesso sarà locale e quindi veloce, se è lontana da Memory l'accesso sarà remoto e quindi lento. Questo ha una conseguenza per SO: quando assegna un frame ad un processo, dovrebbe scegliere di recuperarlo dalla memoria vicina alla CPU in questione.

TRASHING

si verifica quando un processo dispone di meno frame rispetto alle pagine che utilizza frequentemente. Richiede una pag non residente, si verifica un PF. Carica una pag eliminandone un'altra, poco dopo necessita proprio della pag eliminata e si verifica un nuovo PF. Il sistema sta passando + tempo a trasferire pag tra RAM e disco che a eseguire le istruzioni. Pochi frame, molti fault, molto I/O, basso uso di CPU. Se aumentassimo il numero di processi, aumenterebbe l'utilizzo della CPU: mentre un processo attende, un altro può essere eseguito. Però se si supera una certa soglia i frame vengono troppo divisi tra i processi e questi hanno poca memoria a disposizione, generano quindi molti PF. Quasi tutti i processi aspettano il disco e quindi uso CPU crolla. Se aggiungiamo altri processi, le cose peggiorano ancora...

Il DEMAND PAGING funziona bene grazie alla LOCALITÀ: in un intervallo di tempo, un processo tende ad utilizzare ripetutamente un insieme limitato di pag. Se i frame contengono la località corrente, dopo i primi fault gli accessi diventano hit.

Perché avviene il thrashing?

Indicando con L_i la località corrente del processo i , il thrashing si verifica quando:

$$\sum_i |L_i| > \text{numero totale di frame disponibili}$$

Cioè, la somma delle pagine attualmente necessarie ai processi è maggiore della RAM disponibile.

Il sistema non riesce a mantenere tutte le località in memoria: caricare le pagine di un processo significa rimuovere quelle necessarie a un altro.

Per LIMITARE THRASHING possiamo: ridurre n. Processi attivi, usare sostituzione locale, dare priorità ai processi + importanti, assegnare + frame ai processi.

WORKING SET

= l'insieme delle pagine che un processo sta utilizzando nella sua località corrente.

Working set cambia nel tempo perché il processo passa da una località all'altra. Quando si entra in una nuova località, accede a pag non residenti, si ha un picco di PF, le pag nuove vengono caricate e gli accessi successivi sono HIT e il fault rate <<.

Si considera una finestra Δ , espressa come numero degli ultimi riferimenti alla memoria:

$$WSS_i(t, \Delta) = \{\text{pagine riferite dal processo } P_i \text{ negli ultimi } \Delta \text{ accessi}\}$$

La dimensione $|WSS_i|$ indica quanti frame servono al processo per mantenere in RAM la località corrente.

Esempio: se negli ultimi 10 riferimenti abbiamo:

2, 6, 1, 5, 7, 7, 7, 5, 1, 6

allora:

$$WSS = \{1, 2, 5, 6, 7\}, \quad |WSS| = 5$$

Ci interessa solo il numero di PAGINE DISTINTE (le ripetizioni non contano).

Usiamo:

$$\Delta = 6 \text{ riferimenti}$$

Il timer scatta ogni 3 riferimenti; per ogni pagina conserviamo 2 bit:

[intervallo precedente, intervallo attuale]

Reference string divisa in intervalli:

1, 2, 1 | 2, 3, 2 | 3, 4, 3 | 4, 4, 4
 I_1 I_2 I_3 I_4

Fine intervallo	Pagine usate	Contatori [prec., att.]	Working set
I_1	{1,2}	1:01, 2:01, 3:00, 4:00	{1,2}
I_2	{2,3}	1:10, 2:11, 3:01, 4:00	{1,2,3}
I_3	{3,4}	1:00, 2:10, 3:11, 4:01	{2,3,4}
I_4	{4}	1:00, 2:00, 3:10, 4:11	{3,4}

Caso significativo: alla fine di I_3 , la pagina 1 diventa 00, perché non è stata usata negli ultimi due intervalli, quindi **esce dal working set**.

Regola:

$$\text{contatore} \neq 00 \Rightarrow \text{pagina nel working set}$$

Scelta di DELTA

- troppo piccolo: non comprende tutta la località;
- troppo grande: comprende più località vecchie, rischio di uscire dalla memoria
- : comprende tutte le pagine usate dal programma.

Quindi deve approssimare la durata della località corrente.

CASO IDEALE = avere tutto il working set in memoria.

La richiesta complessiva è:

$$D = \sum_i |WSS_i|$$

Se:

$$D \leq m$$

con m numero totale di frame disponibili, tutte le località possono stare in RAM.

Se invece:

$$D > m$$

la memoria non basta e si rischia il **thrashing**. Il sistema deve sospendere o eseguire lo swap-out di uno o più processi, liberando i relativi frame.

PROBLEMA: abbiamo sliding window perché questo algoritmo viene attivato sia quando ho PF che quando ho già la pag nel frame. Tipo: 2,6,1,5,7,7,7,7,5,1,6,2 → rischio di perdere il 2 iniziale e di avere un PF, ma poi mi riserverà. Inoltre per ogni pagina devo tenere traccia del tempo di ultimo accesso, è complicato, quindi uso strategie approssimate (uso timer + reference bit).

(+ il programma è LOCALE e - frame uso).

PAGE FAULT FREQUENCY

Il **PFF** controlla direttamente la frequenza dei page fault, invece di calcolare il working set. La regola è: + frame → - page fault.



- SOPRA la soglia → abbiamo troppi pf e quindi dobbiamo assegnare frame
- SOTTO la soglia → pochi PF e quindi dobbiamo sottrarre frame

Nel mezzo abbiamo l'allocazione corretta

L'algoritmo viene eseguito ad ogni PF. Si misura:

τ = tempo trascorso dal page fault precedente

E viene confrontato con una soglia C.

Caso 1: $\tau < C$

I fault sono molto ravvicinati, quindi la frequenza è alta:

- il processo ha pochi frame;
- gli viene assegnato un nuovo frame;
- il working set viene ampliato.

Caso 2: $\tau \geq C$

È trascorso molto tempo dall'ultimo fault, quindi la frequenza è bassa:

- il processo possiede probabilmente più frame del necessario;
- le pagine con reference bit $R = 0$ non sono state utilizzate nell'intervallo e vengono rimosse dal resident set;
- i reference bit delle pagine mantenute vengono azzerati per iniziare una nuova misurazione.

Quindi:

τ piccolo $\Rightarrow$ aggiungi frame

τ grande $\Rightarrow$ recupera frame inutilizzati

- soglia $C = 4$;
- resident set iniziale: $\{1, 2\}$;
- reference bit iniziali a 0 ;
- fault precedente al tempo $t = 0$.

Reference string:

1, 3, 1, 4, 1, 1, 1, 1, 5, 1, 6

Tempo e riferimento	Situazione	τ	Azione
$t = 1 : 1$	Hit $\rightarrow R_1 = 1$	—	Nessuna
$t = 2 : 3$	Fault su 3	$2 < C$	Fault frequenti: aggiungo frame 3
$t = 3 : 1$	Hit $\rightarrow R_1 = 1$	—	Nessuna
$t = 4 : 4$	Fault su 4	$4 - 2 = 2 < C$	Aggiungo frame 4
$t = 5 \dots 9 : 1$	Solo pagina 1 utilizzata	—	$R_1 = 1$; pagina 2 resta $R_2 = 0$
$t = 10 : 5$	Fault su 5	$10 - 4 = 6 \geq C$	Fault poco frequenti: elimino le pagine con $R = 0$, quindi pagina 2
$t = 11 : 1$	Hit $\rightarrow R_1 = 1$	—	Nessuna
$t = 12 : 6$	Fault su 6	$12 - 10 = 2 < C$	Fault frequenti: aggiungo un frame per 6

I due casi fondamentali sono:

- fault a $t = 4$: $\tau < C \rightarrow$ il processo ha pochi frame, quindi ne riceve uno;
- fault a $t = 10$: $\tau \geq C \rightarrow$ il processo ha abbastanza frame; vengono recuperate le pagine con $R = 0$.

Quindi il Working Set osserva quali pagine compaiono negli ultimi Δ riferimenti, mentre il PFF osserva quanto sono distanti i paghe fault.

ALLOCAZIONE MEMORIA KERNEL

Il kernel deve allocare 2 tipi di memoria: blocchi di pag fisiche CONTIGUE, oppure PICCOLI OGGETTI KERNEL (tipo struct proc, fd, inode)...

Per questo si usano 2 allocatori principali:

- **BUDDY SYSTEM** → gestisce la memoria fisica in blocchi contigui le cui dimensioni sono potenze di 2: pagine. Quando arriva una richiesta, la dimensione viene arrotondata alla potenza di 2 immediatamente superiore.

Per esempio, per una richiesta di 21 KB vengono allocati 32 KB. Se è disponibile un blocco da 256 KB, questo viene diviso ripetutamente: $256 \rightarrow 128+128 > 64+64 \rightarrow 32+32$

Uno dei blocchi da 32 KB viene assegnato alla richiesta, mentre gli altri restano nelle rispettive liste dei blocchi liberi. Due blocchi ottenuti dalla stessa divisione sono chiamati **buddy**. Quando un blocco viene liberato, può essere ricomposto con il proprio buddy: $32+32 \rightarrow 64$. La ricomposizione è possibile solo se i due blocchi:

- hanno la stessa dimensione;
- sono entrambi liberi;
- derivano dalla stessa divisione.

Se anche il buddy del blocco ricomposto è libero, la fusione può continuare verso dimensioni maggiori.

Vantaggio: divisione, allocazione e ricomposizione dei blocchi molto rapide.

Svantaggio: frammentazione interna, perché la richiesta viene arrotondata. Per esempio, allocando 32 KB per una richiesta di 21 KB, si spreca 11 KB.

- **SLAB ALLOCATOR:** gestisce l'allocazione delle strutture dati del kernel, come inode, file descriptor, process control block ed entry delle page table. La sua organizzazione: Cache → Slab → Oggetti kernel

Uno slab è formato da una o più pagine fisicamente contigue, normalmente ottenute dal Buddy System. Ogni slab viene suddiviso in oggetti della stessa dimensione e dello stesso tipo. Una **cache** contiene uno o più slab ed è dedicata a un unico tipo di oggetto kernel. Per esempio, esisteranno cache separate per inode, file descriptor e strutture di processo. Quando viene creata una cache, i suoi oggetti sono marcati come free. Quando il kernel richiede un oggetto: lo slab allocator cerca oggetto libero nella cache corrispondente, lo marca come used, lo restituisce al kernel. Quando non serve più viene restituito alla stessa cache e marcato come free così può essere riutilizzato. Se non sono disponibili oggetti liberi, lo Slab Allocator richiede nuove pagine al Buddy System e crea un nuovo slab.

VANTAGGI: allocazione/deallocazione rapide, riutilizzo di oggetti già predisposti, < frammentazione interna per strutture kernel ripetute, no costo per inizializzazione completa ad ogni nuova richiesta.

PREPAGING = carica anticipatamente alcune pagine prima che vengano effettivamente richieste, per ridurre i page fault iniziali.

a = frazione di pag effettivamente usata

s = pag precariate.

Pag utili = $s \cdot a$

Pag inutilmente caricate = $s \cdot (1 - a)$

Se a è vicino a 1, il prepaging è molto conveniente perché evitiamo PF, ma se è vicino a 0, stiamo sprecaendo molta memoria e I/O.

Conviene solo: **costo_page_fault_evitati > costo_pag_caricate_inutilmente**

Dimensione pag

pagine piccole $\Rightarrow$ più pagine $\Rightarrow$ più possibili page fault

pagine grandi $\Rightarrow$ meno pagine $\Rightarrow$ meno page fault, ma ciascuno più costoso

Il numero preciso di page fault dipende però dalla reference string, dalla località e dal numero di frame disponibili.

Aspetto	Pagine piccole	Pagine grandi
Frammentazione interna	Minore	Maggiore
Numero di pagine del processo	Maggiore	Minore
Dimensione page table	Maggiore: servono più entry	Minore: servono meno entry
Numero di page fault	Tendenzialmente maggiore	Tendenzialmente minore
Costo del singolo page fault	Minore: si trasferiscono meno dati	Maggiore: si trasferiscono più dati
I/O complessivo	Più trasferimenti piccoli	Meno trasferimenti, ma più grandi
Località	Caricamento più preciso	Può caricare dati vicini inutilizzati
TLB reach	Minore	Maggiore
Numero di entry TLB necessarie	Maggiore	Minore
Spazio virtuale coperto da una pagina	Minore	Maggiore

TLB REACH

TLB Reach = numero di entry TLB $\times$ dimensione pagina

Ogni TLB entry contiene la traduzione di una pag virtuale

Idealmente avremmo: $TLB\ Reach \geq Working\ Set$ così tutte le traduzioni delle pag usate frequentemente dal processo rimangono in TLB. Se $working_set > TLB_REACH$ allora abbiamo TLB MISS che aumentano. Se aumentano TLBMISS non è detto anche che necessariamente aumentino PF, perché la pag può essere presente in RAM anche se assente in TLB.

Soluzione	Vantaggio	Svantaggio
Aumentare le entry della TLB	Più traduzioni memorizzate	Hardware più costoso e complesso
Aumentare la page size	Ogni entry copre più memoria	Maggiore frammentazione interna
Usare più page size	Pagine grandi solo dove convenienti	Gestione hardware e software più complessa

I/O INTERLOCK = meccanismo che coordina page replacement con operazioni di I/O, impedendo la sostituzione delle pag coinvolte.

Quando avviene un'operazione di I/O il dispositivo trasferisce i dati verso o da un preciso buffer in RAM. Se con il page replacement rimuovesse la pag dal buffer mentre sta ancora trasferendo i dati, il dispositivo potrebbe leggere/scrivere nel frame sbagliato. Quindi si applica il pinning: SO blocca la pag nel frame fisico, la esclude dalle possibili vittime, esegue op di I/O e alla fine rimuove il blocco.

